

Vibe Coding using Claude Code

Workflow: Explore → Plan → Code → Commit

Context Engineering

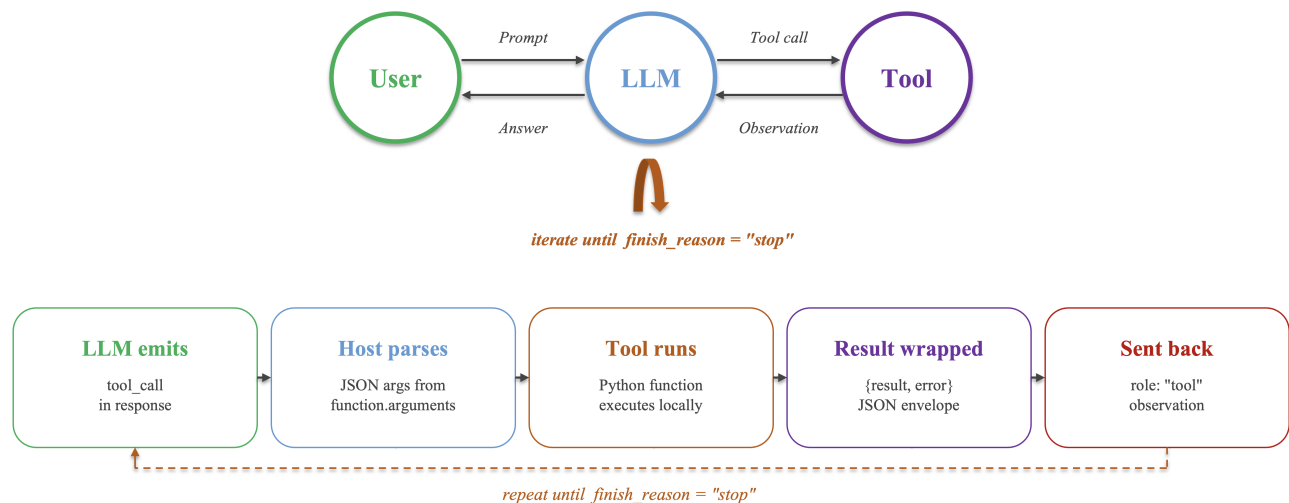
Skills - Subagents - MCP - Hooks - Custom Slash Commands

Nguyễn Quốc Thái

Đinh Quang Vinh

I. Giới thiệu

Ngày nay, mô hình ngôn ngữ lớn (LLM) đã thay đổi căn bản cách lập trình viên làm việc. Thay vì gõ từng dòng, chúng ta giờ đây có thể mô tả ý định bằng ngôn ngữ tự nhiên và để AI agent đề xuất diff (tức là AI đề xuất việc thay đổi code dưới dạng *+ dòng thêm* / *- dòng xoá* để lập trình viên kiểm duyệt trước khi chỉnh sửa), gọi tool, chạy test, và đề xuất commit message. **Andrej Karpathy** gọi pattern làm việc này là *vibe coding*: “you fully give in to the vibes... see stuff, say stuff, run stuff, copy paste stuff”. Khái niệm này tóm gọn một thay đổi: từ viết code sang điều khiển agent viết code.



Hình 1: Pipeline tổng quan của Vibe Coding với Claude Code.

Trong phần nội dung này, chúng ta sẽ tìm hiểu toàn bộ pipeline vibe coding với Claude Code – một CLI agent loop chuyên dụng của Anthropic:

- **Claude Code**: agent loop trên terminal, có sẵn các tool Read/Write/Edit/Bash/Grep, hỗ trợ MCP, Subagent, Skill, Hook, Slash Command.
- **Workflow E→P→C→C**: bốn bước chuẩn cho mọi quy trình vibe coding – Explore, Plan, Code, Commit.
- **Context Engineering**: quản lý context window hiệu quả.
- **ReAct Agent Project**: xây dựng một ReAct Agent từ đầu bằng quy trình vibe coding (Lab 1), sau đó nâng cấp với 4 cơ chế mở rộng (Lab 2).

Sau đó chúng ta sẽ triển khai ReAct Agent thông qua OpenAI Function Calling và httpx (cho Tavily REST API), đồng thời sử dụng Claude Code làm môi trường vibe coding.

Mục lục

I.	Giới thiệu	1
II.	Vibe Coding	5
II.1.	Định nghĩa Vibe Coding	5
II.2.	So sánh với coding truyền thống	6
III.	Hệ sinh thái Claude	7
III.1.	Claude Code là gì?	8
III.2.	Claude Code = Agent Loop chuyên dụng	9
III.3.	Tool Call Lifecycle (OpenAI Function Calling)	11
IV.	Setup & Workspace – Cài đặt Claude Code	14
IV.1.	Quy trình cài đặt sáu bước	14
IV.2.	Tích hợp IDE với VSCode	15
IV.3.	Permissions trong Claude Code	16
IV.4.	Slash Command Reference	17
V.	Workflow	
	Explore → Plan → Code → Commit	19
V.1.	Tổng quan workflow E→P→C→C	19
V.2.	Phase 1 – Explore (Khám phá)	19
V.3.	Phase 2 – Plan (Lập kế hoạch)	20
V.4.	Phase 3 – Code (Sinh diff)	20
V.5.	Phase 4 – Commit (Sign-off)	21
V.6.	Quản lý bộ nhớ với CLAUDE.md – Memory	22
V.7.	@file references và !shell inline	23
VI.	Lab 1 – Build A ReAct Agent using Claude Code	24
VI.1.	Mục tiêu Lab 1	24
VI.2.	Kiến trúc ReAct Agent	24
VI.3.	Vibe coding Lab 1 sử dụng Claude Code	26
VI.4.	Tool Registry – tools.py	35
VI.5.	LLM Client – llm.py	38

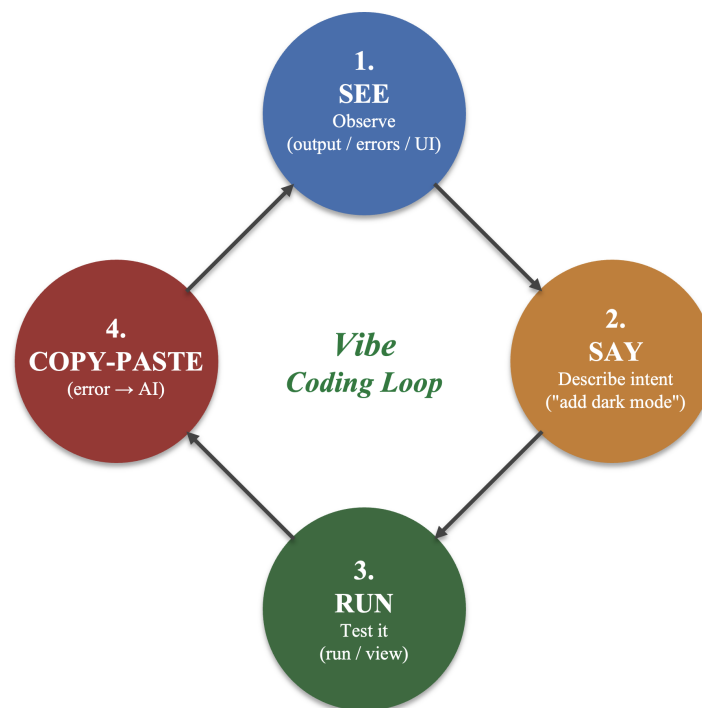
VI.6.	Agent Loop – <code>agent.py</code>	39
VI.7.	CLI Entry Point – <code>main.py</code>	41
VI.8.	Unit Tests – <code>tests/test_tools.py</code>	41
VI.9.	Smoke Tests	43
VII.	Context Engineering	44
VII.1.	Context Window 200K-1000K	44
VII.2.	Lệnh <code>/context</code> và <code>/status</code>	45
VII.3.	<code>/compact</code> vs <code>/clear</code> vs Auto-compact	45
VII.4.	Kiểm soát ngữ cảnh như thế nào?	46
VII.5.	Bốn workflow pattern nâng cao	47
VIII.	Mở rộng Claude Code:	
	Skill, Subagent, MCP, Hook, Custom Slash Command	52
VIII.1.	Tổng quan các cơ chế mở rộng	52
VIII.2.	Skills – Workflow Auto-trigger	53
VIII.3.	Subagents – Chuyên biệt với Context Riêng	54
VIII.4.	MCP – Model Context Protocol	56
VIII.5.	Hooks – Event-driven Automation	57
VIII.6.	Custom Slash Commands	58
IX.	Lab 2 – Mở rộng ReAct Agent với MCP, Subagent, Skill	60
IX.1.	Mục tiêu Lab 2	60
IX.2.	Phase 1 – Web search với Tavily REST API	61
IX.3.	Phase 2 – Subagent <code>agent-reviewer</code>	63
IX.4.	Phase 3 – Skill <code>react-loop-debug</code>	65
IX.5.	Phase 4 – Build MCP Server (Agent-as-a-Service)	67
IX.6.	Integration Test End-to-End	71
X.	Câu hỏi trắc nghiệm	72

II. Vibe Coding

Trước khi đi vào cài đặt cụ thể, workflow chi tiết, hay xây dựng ReAct Agent, chúng ta sẽ tìm hiểu cơ bản về các nội dung: vibe coding nghĩa là gì, có khác gì so với coding truyền thống, công cụ nào thực hiện điều này, và vị trí của Claude Code trong hệ sinh thái rộng hơn của Anthropic.

II.1. Định nghĩa Vibe Coding

Vibe Coding – tạm dịch là “lập trình theo *vibe*” – là pattern hợp tác giữa người lập trình và AI agent, trong đó con người dẫn dắt ý định và phản biện, còn AI đảm nhận phần sinh diff, gọi tool, chạy lệnh. Khái niệm được Andrej Karpathy phổ biến đầu năm 2025 trong một loạt bài viết và post trên X (Twitter), mô tả trạng thái khi developer tận dụng tài nguyên của AI để tự động hoá quá trình lập trình: *nhìn (see)* output, *nói (say)* ý định mới, *chạy (run)* thử, và *copy-paste* thông báo lỗi ngược lại cho AI. Bốn động từ này tạo thành một vòng lặp khép kín giúp tăng khả năng suy luận và hoàn thiện các tác vụ lập trình phức tạp:



Hình 2: Vibe Coding Loop: *See* → *Say* → *Run* → *Copy-paste*. Bốn bước này lặp lại liên tục, developer giữ vai trò orchestrator của vòng lặp thay vì người gõ code.

Điểm quan trọng cần làm rõ ngay từ đầu: vibe coding khác với hình dung phổ thông về AI viết hộ code. Đây không phải là một mệnh lệnh một chiều mà là một *vòng lặp hai chiều*. Developer không đơn thuần nhận output rồi paste vào file; trái lại, họ giữ ba vai trò liên tục đan xen, mỗi vai trò đòi hỏi một loại tư duy khác:

- **Architect (Kiến trúc sư):** đặc tả *cái gì* và *vì sao*; chia bài toán thành các bước AI có thể thực hiện.

- **Reviewer (Người duyệt):** đọc diff, kiểm tra logic, đối chiếu quy tắc, đảm bảo chất lượng trước khi accept.
- **Integrator (Người tích hợp):** nối output của AI vào mã nguồn, chạy test, xử lý xung đột với phần code đã có.

Ba vai trò này không tách rời mà hỗ trợ liên tục, developer có thể chuyển qua lại nhiều lần giữa architect (khi lên plan), reviewer (khi đọc diff do Claude đề xuất), và integrator (khi chạy test và xử lý lỗi). Đây là điểm khác biệt căn bản so với việc gõ code tuần tự: thay vì tập trung sâu vào một file trong nhiều phút, developer giờ liên tục thao tác trên nội dung file.

II.2. So sánh với coding truyền thống

Để hiểu rõ hơn về vibe coding và coding truyền thống, bảng dưới so sánh hai cơ chế làm việc trên năm trục quan trọng, từ chủ thể chính, vai trò AI, đơn vị làm việc, tốc độ, đến loại rủi ro chủ yếu:

Khía cạnh	Coding truyền thống	Vibe Coding
Primary actor	Developer gõ từng dòng	Developer đặc tả, AI sinh, developer duyệt
Vai trò AI	Autocomplete cấp token (Copilot)	Agent loop cấp task (Claude Code)
Đơn vị làm việc	Token / dòng	Task / commit
Tốc độ	Tuyến tính theo skill	Nhanh hơn 3–10× trên việc routine
Rủi ro chính	Bug do gõ sai	Bug do mô tả ý định không rõ
Phù hợp nhất với	Thuật toán mới, code tối ưu hiệu năng	Code lặp khuôn, refactor, các tính năng thông thường

Một cách hiểu khác chúng ta có thể thấy từ đây: vibe coding chuyển kỹ năng từ *viết* sang *mô tả* và *đánh giá*. Developer giỏi vibe coding là người mô tả ý định rõ ràng, đọc diff nhanh, và biết khi nào nên dừng AI để tự code (ví dụ những đoạn cần optimize sâu hoặc thuật toán đặc thù). Nội dung tiếp theo chúng ta trao đổi là về sự thay đổi và phát triển trong AI coding tools. Từ autocomplete cấp token đến agent loop đa bước, chúng ta đã đi qua một số cột mốc lớn:

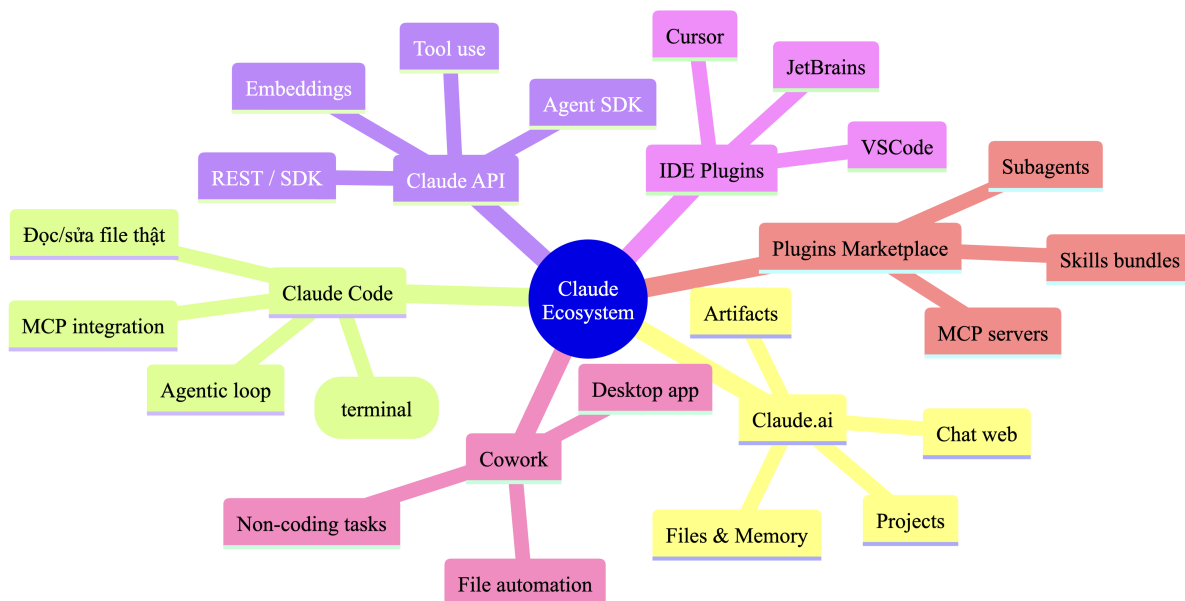


- **2021 – GitHub Copilot:** autocomplete cấp token, gợi ý dòng kế tiếp ngay trong IDE. AI hoàn toàn thụ động, chỉ đoán chuỗi token kế tiếp dựa trên context xung quanh.

- **2022 – ChatGPT**: chat Q&A về code, AI có thể giải thích, sinh đoạn code đứng riêng, nhưng nằm ngoài IDE – developer phải copy-paste qua lại.
- **2023 – Cursor IDE**: chat + inline edit ngay trong editor, AI bắt đầu sửa nhiều dòng cùng lúc dưới sự kiểm soát của developer.
- **2025 – “Vibe coding”**: thuật ngữ do Andrej Karpathy phổ biến, mô tả workflow developer trở thành *orchestrator* của agent thay vì người gõ code.
- **2025 – Claude Code**: agent loop trong terminal, AI tự gọi tool (read file, run command, edit) liên tục cho đến khi đạt mục tiêu.

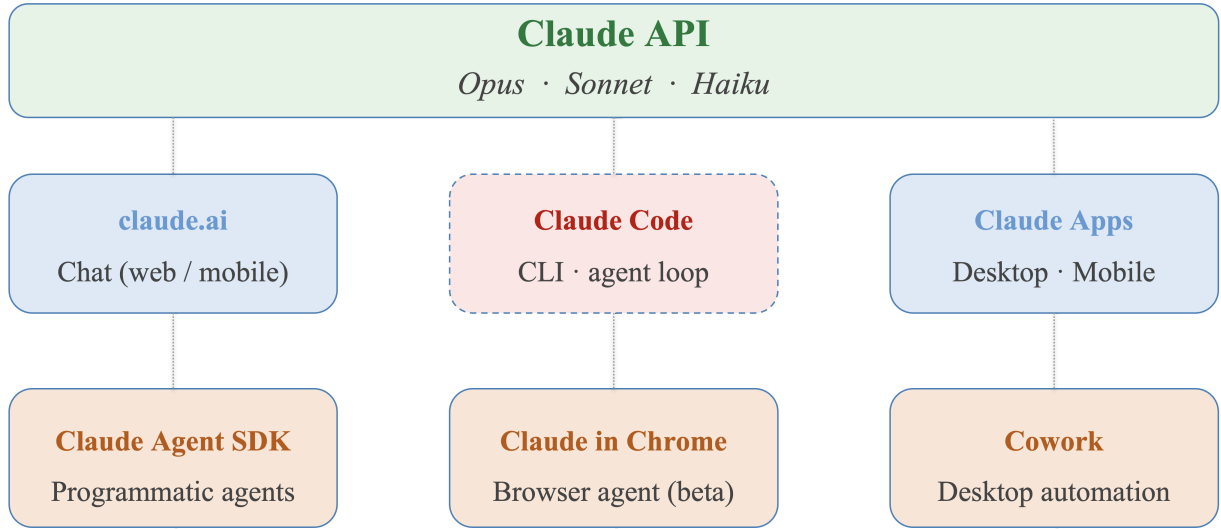
III. Hệ sinh thái Claude

Khi đã hiểu lịch sử của AI coding tools, chúng ta sẽ tìm hiểu về một công cụ điển hình cho vibe coding là **Claude Code**. Trước tiên sẽ tìm hiểu về hệ sinh thái của Claude.



Hình 4: Hệ sinh thái Claude.

Hệ sinh thái Claude gồm nhiều thành phần như Hình 4, tuy nhiên cơ bản được tổ chức thành ba tầng theo mức độ trừu tượng. Tầng dưới cùng là **Claude API** – backend HTTP cung cấp các model Opus, Sonnet, Haiku; mọi sản phẩm phía trên đều cuối cùng gọi đến API này. Tầng giữa là **Surfaces** (claude.ai, Claude Code, Claude Apps) – ba giao diện phục vụ ba use case khác nhau: chat web, agent loop terminal, và tự động hoá desktop. Tầng trên cùng là **Nâng cao** (Agent SDK, Claude in Chrome, Cowork) – các tích hợp chuyên biệt cho phát triển agent programmatic, tự động hoá browser, và tự động hoá file.



Hình 5: Các tầng hoạt động của Claude – Claude API ở trên, ba surfaces ở giữa, ba sản phẩm nâng cao ở dưới.

Sơ đồ trên cho thấy luồng dữ liệu giữa các tầng. Khi developer gõ lệnh trong Claude Code (tầng Surfaces), prompt được gửi xuống Claude API; API trả về phản hồi chứa tool calls; Claude Code thực thi tool, gửi observation về API, lặp lại đến khi LLM trả về `finish_reason = "stop"`. Toàn bộ vòng lặp diễn ra trong vài giây mà developer không phải nghĩ đến chi tiết kỹ thuật.

- **Tầng API:** Claude API với các model Opus 4.7, Opus 4.6, Sonnet 4.6, Haiku 4.5, ... – nguồn giao tiếp trực tiếp với các models.
- **Tầng Surfaces:** `claude.ai` (chat web/mobile), **Claude Code** (CLI / agent loop), và Claude Apps (desktop, mobile).
- **Tầng Nâng cao:** Claude Agent SDK (programmatic agents), Claude in Chrome (browser agent, beta), Cwork (tự động hoá desktop).

Khi nói “vibe coding”, chúng ta chủ yếu thao tác ở **tầng Surfaces – Claude Code**. Tầng API là phần động cơ lõi model (model nào được gọi), tầng Advanced là các pattern mở rộng (agent programmatic, browser, tự động hoá).

III.1. Claude Code là gì?

Trước khi đi sâu vào agent loop, ta tìm hiểu về Claude Code. Bảng dưới đối chiếu sáu sản phẩm chính và so sánh với `claude.ai`, Claude Code, Claude API, và các surface khác:

Sản phẩm	Mô tả	Khi nào dùng
Claude API	Backend HTTP – Opus / Sonnet / Haiku.	Khi xây ứng dụng riêng, gọi model trực tiếp.
claude.ai	Chat web/mobile thuần.	Hỏi đáp, brainstorm, soạn tài liệu.
Claude Code	CLI agent loop.	<i>Vibe coding</i> – viết, sửa, refactor, debug code.
Claude Apps	Desktop / mobile UI.	Workflow ngoài terminal – soạn email, đọc file.
Agent SDK	SDK programmatic.	Xây agent custom với Python/TypeScript.
Claude in Chrome	Browser agent (beta).	Tự động hoá thao tác web – form, scrape, click.
Cowork	Tự động hoá desktop.	Quản lý file, lịch trên máy người dùng.

Claude Code là *surface* chuyên cho coding workflow

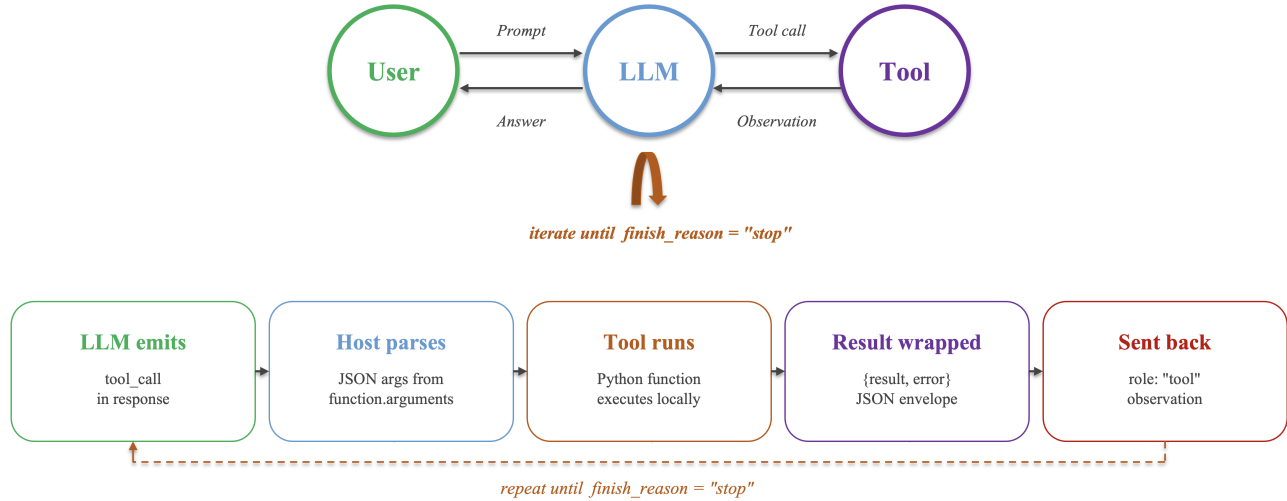
Tất cả các sản phẩm trên đều dùng chung Claude API ở dưới. Khác biệt nằm ở *surface* – giao diện và toolkit. Claude Code chọn terminal làm surface vì lý do thực dụng: terminal là môi trường nguyên gốc của file system, git, package manager, test runner. Mọi tool mà developer cần (đọc file, chạy lệnh, grep, web fetch) đều có sẵn dưới dạng built-in tool, không phải clone từ web hay copy-paste qua lại.

III.2. Claude Code = Agent Loop chuyên dụng

Khi đã hiểu rõ về Claude Code, tiếp theo ta xem nó hoạt động như thế nào. Nắm vững mô hình agent loop sẽ giúp chúng ta hiểu và sử dụng hiệu quả các tính năng còn lại (tool, permission mode, context window, extension mechanism).

Claude Code không phải là chatbot. Nó là một *agent loop* chạy trong REPL terminal – một quá trình lặp gồm năm bước mà LLM, host, và file system thay phiên nhau đóng vai trò:

- **Nghe prompt:** developer gõ yêu cầu (vd: “thêm endpoint /healthz vào server.py”).
- **Quyết định tool:** LLM phân tích prompt, chọn tool nào sẽ gọi (Read, Write, Edit, Bash, Grep, WebFetch, ...).
- **Thực thi:** tool chạy, kết quả được đóng gói lại.
- **Quan sát:** LLM nhận output, quyết định tiếp tục gọi tool hay trả về câu trả lời cuối.
- **Dừng:** khi LLM trả về “stop” (không còn tool call), loop kết thúc.



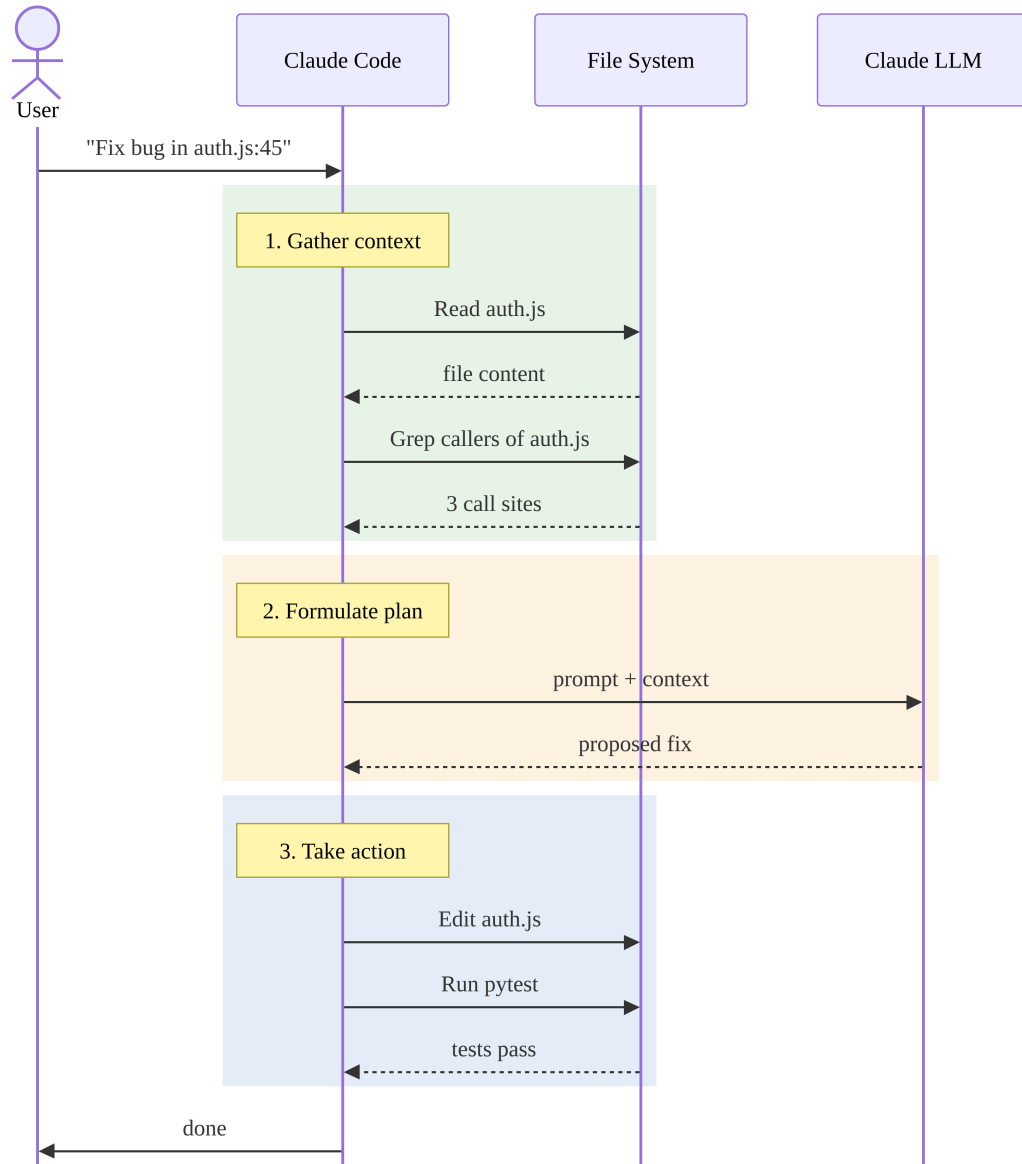
Hình 6: Pipeline tổng quan của Vibe Coding với Claude Code.

Khác biệt then chốt so với chat

Trong chat, LLM chỉ quyết định *nói gì*. Trong Claude Code, LLM quyết định *làm gì*. Sự khác biệt này cho phép thực thi mọi tính năng đặc trưng của agent: tool calling, multi-step planning, terminating condition, error handling.

Vòng lặp năm bước ở trên mô tả mặt *kỹ thuật* của agent. Ở mức độ đơn giản, ta có thể gom thành ba bước dưới góc độ xem xét mô hình AI làm gì: Claude Code thực hiện **Gather** (đọc files, hiểu bối cảnh), **Plan** (đề xuất thay đổi), và **Act** (thực thi diff, chạy test).

Để hiểu Claude Code mở rộng được như thế nào, cần nhìn vào kiến trúc bốn tầng của nó. Mỗi tầng có vai trò riêng biệt, và việc tách rõ giúp hệ sinh thái có thể bổ sung tool, đổi LLM, hay thêm extension mà không phá vỡ phần còn lại:



Hình 7: Sơ đồ luồng hoạt động giữa các tầng kiến trúc.

Bốn tầng kiến trúc Claude Code làm việc theo phân vai rõ ràng. **REPL** quản lý tương tác với user (đọc prompt, hiển thị diff); **Agent loop** điều phối vòng lặp ReAct; **Tool layer** cung cấp các built-in tool (Read, Write, Bash, Grep...) và tích hợp với MCP servers; **LLM layer** gọi Claude API. Việc tách rõ bốn tầng cho phép Anthropic thêm tool mới (vd WebFetch), đổi model (Sonnet → Opus), hay thêm cơ chế mở rộng mà không phá vỡ các tầng còn lại – một ví dụ điển hình của sự tách biệt các chức năng trong thiết kế hệ thống.

III.3. Tool Call Lifecycle (OpenAI Function Calling)

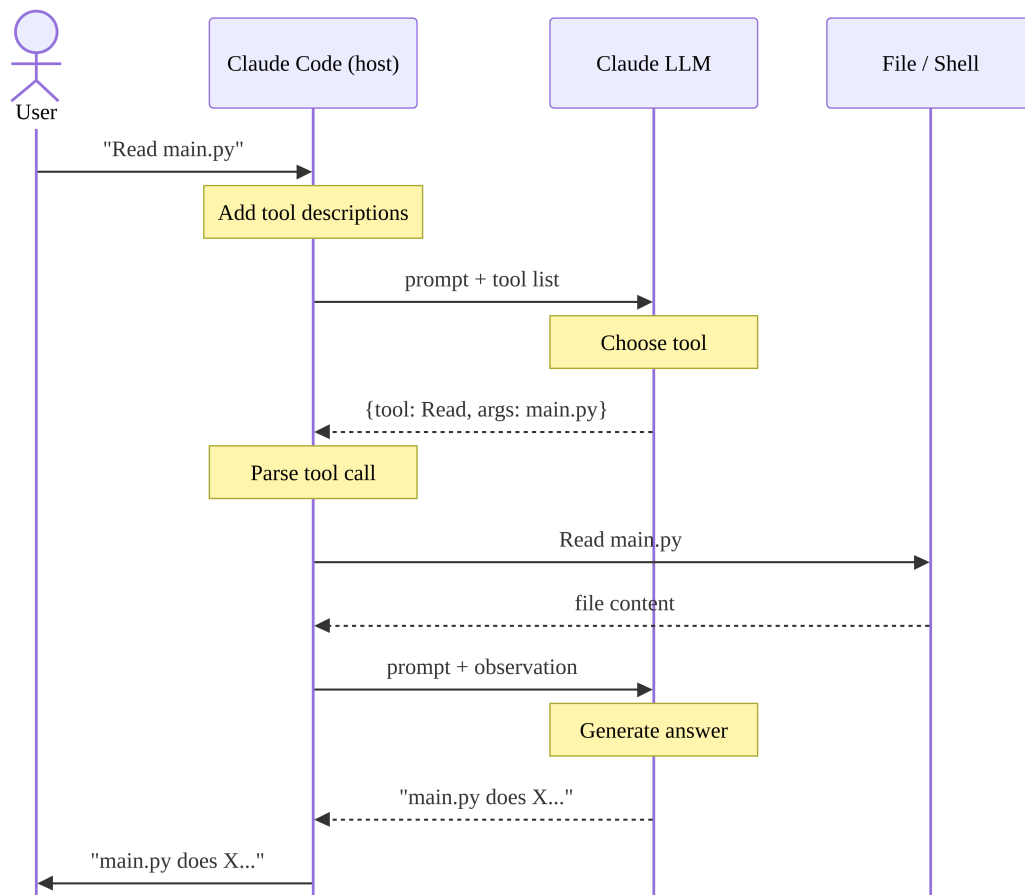
Trong các phần trước, chúng ta đã mô tả agent loop. Một tool call đi qua quá trình như sau:

- **LLM emits tool_call**: response trả về `finish_reason = "tool_calls"`, kèm danh sách tool calls (mỗi cái có `id`, `name`, `arguments` dạng JSON string).
- **Host parse JSON**: code Python parse `arguments` thành dict bằng `json.loads(...)`.
- **Tool runs**: function Python tương ứng thực thi locally với các argument vừa parse.
- **Đóng gói kết quả**: kết quả gói trong envelope `{result, error}` và serialize lại JSON.
- **Gửi lại**: gửi observation về LLM với role `tool` và `tool_call_id` khớp với id ban đầu.

Định dạng tool_call của OpenAI

```
1 # Response từ LLM:
2 {
3   "finish_reason": "tool_calls",
4   "tool_calls": [{
5     "id": "call_abc",
6     "type": "function",
7     "function": {
8       "name": "read_file",
9       "arguments": "{\"path\": \"main.py\"}"
10    }
11  }]
12 }
13
14 # Host gửi lại observation:
15 {
16   "role": "tool",
17   "tool_call_id": "call_abc",
18   "content": "# main.py\nprint('hi')"
```

Quy trình lặp đi lặp lại cho đến khi LLM trả về `finish_reason = "stop"` – lúc đó `content` của message chính là câu trả lời cuối cùng. Đây cũng chính là wire-level protocol mà chúng ta sẽ triển khai cụ thể ở Lab 1, khi ReAct Agent gọi OpenAI API. Quá trình được tổng quát như hình dưới:



Hình 8: Sơ đồ luồng tổng quát về quá trình hoạt động của tool.

Sơ đồ trên tổng hợp các bước của một tool calling thành một flow chart. Đáng chú ý là tính **tách biệt** giữa LLM và file system: model không bao giờ chạm trực tiếp vào file hay shell – mọi thao tác đều phải đi qua host (Claude Code) làm trung gian. Đây là điểm thiết kế quan trọng vì lý do bảo mật: nếu LLM bị prompt injection và yêu cầu xoá file, host vẫn có cơ hội chặn ở bước Approval. Trong Lab 1, chính ta sẽ đóng vai trò host – viết Python code parse tool call, thực thi tool, và gửi observation về cho LLM.

IV. Setup & Workspace – Cài đặt Claude Code

Trước khi đi vào workflow vibe coding chính thức, chúng ta cần một môi trường hoạt động: Claude Code đã cài, API key đã cấu hình, IDE đã tích hợp, và biết ba chế độ permission. Mục này hướng dẫn end-to-end từ cơ bản trống đến prompt đầu tiên, đồng thời giới thiệu các phím tắt và lệnh slash cốt lõi.

IV.1. Quy trình cài đặt sáu bước

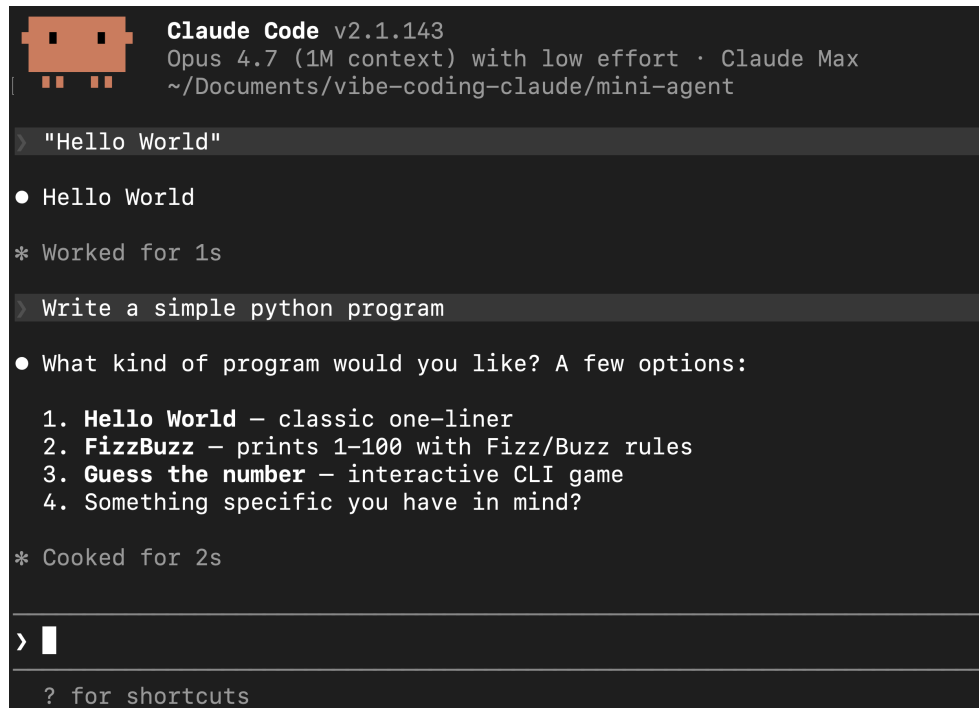
Claude Code được phân phối qua npm registry. Quy trình cài chuẩn gồm sáu bước, mỗi bước chỉ mất 1–2 phút:

1. **Step 1 – Cài Node.js ≥ 18 :** tải từ nodejs.org (file installer cho Windows/macOS) hoặc dùng package manager: `brew install node` (macOS), `sudo apt install nodejs` (Ubuntu). Kiểm tra: `node -version`.
2. **Step 2 – Cài Claude Code:** chạy `npm install -g @anthropic-ai/claude-code`. Cờ `-g` cài vào global path, đảm bảo lệnh `claude` chạy được ở mọi thư mục.
3. **Step 3 – Lấy Anthropic API key:** vào console.anthropic.com, tạo key, sau đó `export ANTHROPIC_API_KEY=sk-ant-...` (bỏ vào `.zshrc`/`.bashrc` cho persistent). Lab 1 dùng OpenAI key thay thế (ReAct Agent triển khai trên Function Calling chuẩn).
4. **Step 4 – Tạo workspace:** `mkdir aio-vibe-coding && cd aio-vibe-coding`. Workspace nên là git repo từ đầu – chạy luôn `git init`, vì CLAUDE.md cần được version control.
5. **Step 5 – Khởi chạy REPL:** gõ `claude`. Nếu thành công, terminal hiển thị banner welcome cùng `cwd`: (current working directory) và prompt `>`.
6. **Step 6 – Prompt đầu tiên:** thử "Hello, list files in this folder". Claude Code sẽ xin phép gọi tool Bash (`ls`), bạn approve, và nó trả về danh sách file.

Lưu ý

Lỗi phổ biến nhất ở Step 2 là `permission denied` trên macOS/Linux. Nguyên nhân: `npm` đang ghi vào thư mục global thuộc `root`. Cách fix sạch: cấu hình lại `npm` prefix sang home directory thay vì dùng `sudo`: `npm config set prefix ~/.npm-global && export PATH=$PATH: ~/.npm-global/bin`.

Sau khi cài đặt xong chúng ta có được giao diện như hình dưới:



```

Claude Code v2.1.143
Opus 4.7 (1M context) with low effort · Claude Max
~/Documents/vibe-coding-claude/mini-agent

> "Hello World"

• Hello World

* Worked for 1s

> Write a simple python program

• What kind of program would you like? A few options:

  1. Hello World – classic one-liner
  2. FizzBuzz – prints 1-100 with Fizz/Buzz rules
  3. Guess the number – interactive CLI game
  4. Something specific you have in mind?

* Cooked for 2s

> 
? for shortcuts

```

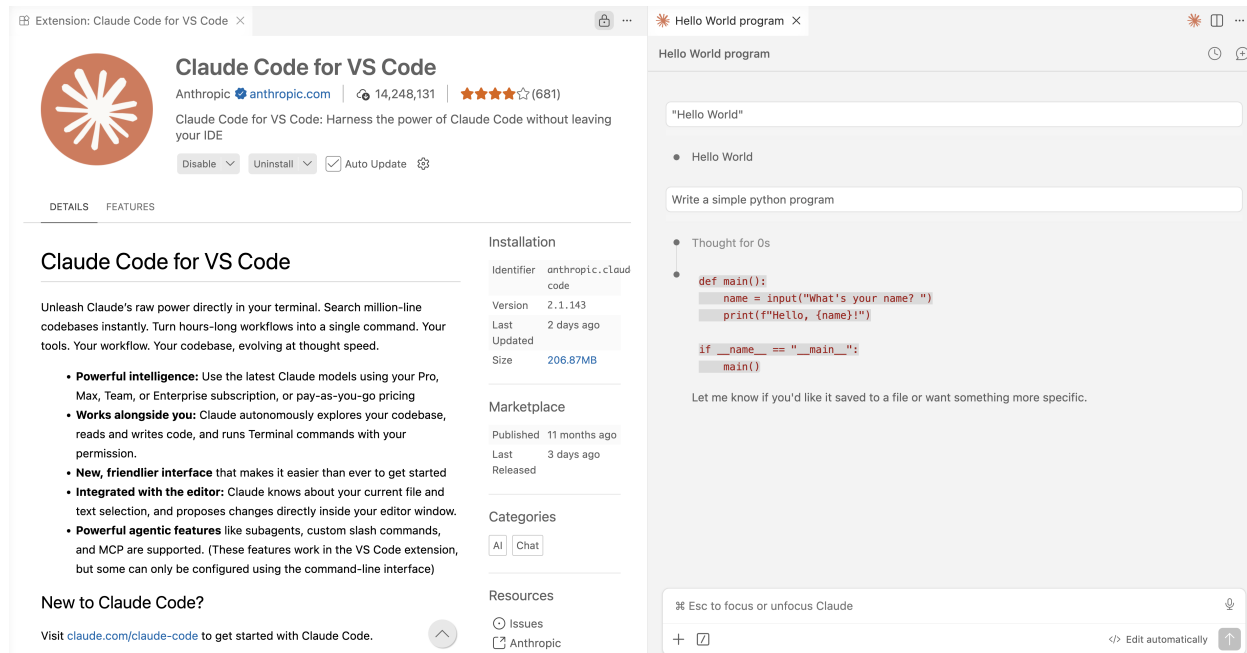
Hình 9: Giao diện làm việc của Claude Code trong terminal.

IV.2. Tích hợp IDE với VSCode

Claude Code chạy trên terminal, nhưng đa số developer làm việc trong IDE. Anthropic cung cấp extension chính thức cho VSCode để hai bên cùng share context: file đang mở trong editor được tự động đưa vào prompt Claude.

- **Cài extension:** VSCode marketplace, tìm “Claude Code” (publisher: Anthropic), bấm Cài đặt.
- **Mở panel:** `Ctrl` + `Shift` + `P` (hoặc `⌘` + `Shift` + `P` trên Mac) → “Claude Code: Open Panel”. Panel hiện như một sidebar bên phải.
- **Inline diff preview:** khi Claude đề xuất sửa file, diff hiện ngay trong editor với highlight xanh/đỏ – bạn duyệt từng hunk bằng phím mũi tên.
- **Accept / reject từng hunk:** `Tab` accept, `R` reject, `Enter` confirm. Không phải accept-all-or-nothing.
- **Keyboard navigation:** `n` next hunk, `p` previous, `q` thoát review.
- **Auto context:** file đang focus được tự động thêm vào context của Claude → không cần `@file.py` thủ công nếu file đó đang mở.

Sau khi cài đặt xong chúng ta có được giao diện như hình dưới:



Hình 10: Giao diện làm việc của Claude Code trong VSCode.

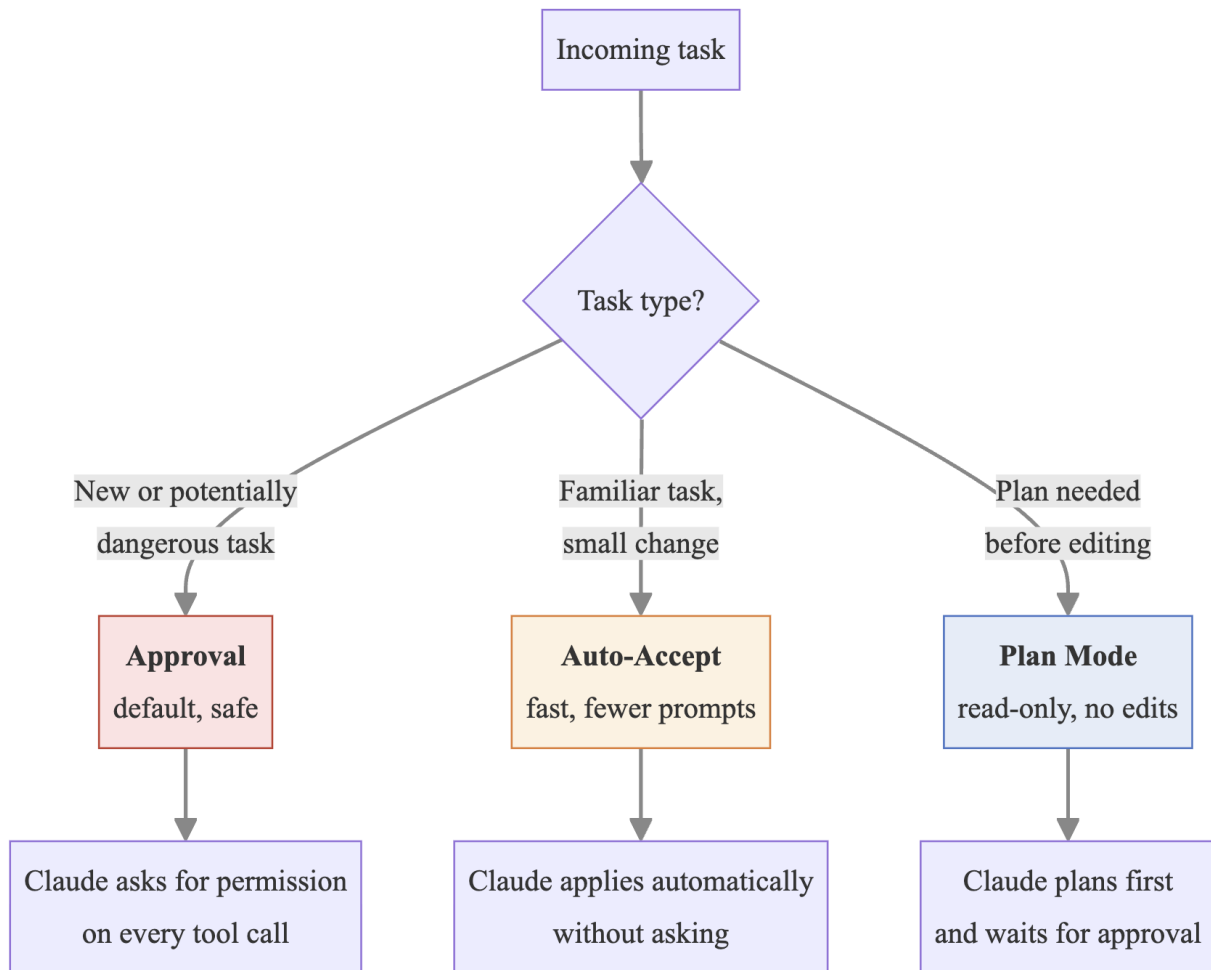
💡 Khi nào dùng VSCode vs terminal thuần

Dùng VSCode khi cần xem diff nhiều, làm việc với UI/frontend (cần preview), hoặc khi mã nguồn lớn nên cần điều hướng nhanh bằng explorer. Dùng terminal thuần khi pair-programming với agent trên server từ xa, làm việc với CLI tools, hoặc khi muốn workflow tối giản.

IV.3. Permissions trong Claude Code

Claude Code quản lý ranh giới giữa AI và file system bằng *permission modes*. Có ba mode, luân chuyển bằng `Shift + Tab`:

Mode	Trigger	Hành vi	Khi nào dùng
Approval	Mặc định	Hỏi xác nhận trước mỗi edit. Chậm nhất, an toàn nhất.	Code sản xuất, repo lạ, tasks rủi ro cao.
Auto-Accept	<code>Shift + Tab</code> × 1	Tin tưởng Claude, tự động áp dụng edits không hỏi.	Routine refactor trong code bạn hiểu rõ.
Plan Mode	<code>Shift + Tab</code> × 2	Không edit. Chỉ đề xuất plan dạng text, chờ duyệt.	Exploration, architecture, brainstorm.



Hình 11: Cây quyết định ba chế độ permission. Bấm **Shift + Tab** để thay đổi giữa chúng.

Permission modes

Ba mode tạo thành một thang trade-off: *tốc độ vs. kiểm soát*. Plan Mode chậm nhất nhưng ít rủi ro (không chỉnh sửa gì). Approval ở giữa: chậm, mỗi action phải duyệt. Auto-Accept nhanh nhất nhưng nguy hiểm nhất. Quy tắc thực tế: mặc định Approval; tăng lên Auto-Accept khi đã làm cùng tính năng 10+ phút và Claude đã chứng tỏ hiểu đúng convention; giảm xuống Plan Mode bất cứ khi nào không chắc về phạm vi.

IV.4. Slash Command Reference

Claude Code cung cấp một bộ lệnh slash để điều khiển session ngoài tương tác chat. Bảng dưới phân loại theo mục đích:

Nhóm	Lệnh	Mô tả
Session Control	<code>/help</code>	Liệt kê tất cả lệnh available.
	<code>/clear</code>	Xoá toàn bộ history, khởi đầu lại.
	<code>/compact</code>	Tóm tắt history thành digest, giữ N turn cuối.
	<code>/exit or Ctrl + D</code>	Thoát REPL.
Power Features	<code>@file.py</code>	Inline reference: nạp nội dung file vào context.
	<code>!command</code>	Chạy shell command, thu thập output vào context.
	<code>/mcp</code>	List MCP servers đang connect.
	<code>/agents</code>	List subagents available trong project.

Hai lệnh `@file` và `!shell` đặc biệt quan trọng vì chúng biến mọi prompt thành mini-IDE – chi tiết sẽ được trình bày ở Mục V (sau khi học CLAUDE.md). Còn `/mcp` và `/agents` cách để tăng khả năng mở rộng hình vi, chức năng của agent, sẽ được giới thiệu ở Mục VII.

💡 Workflow cho project đầu tiên

- (1) `claude` mở REPL;
- (2) hỏi tổng quan: “Read README.md and summarize the project in 5 bullets”;
- (3) bấm `Shift` + `Tab` ×2 vào Plan Mode;
- (4) mô tả tính năng muốn thêm, yêu cầu plan;
- (5) review plan;
- (6) `Shift` + `Tab` thoát Plan Mode, để Claude code;
- (7) review từng diff;
- (8) `!pytest` và iterate;
- (9) `!git diff + commit` message nếu muốn đẩy code lên github;
- (10) commit.

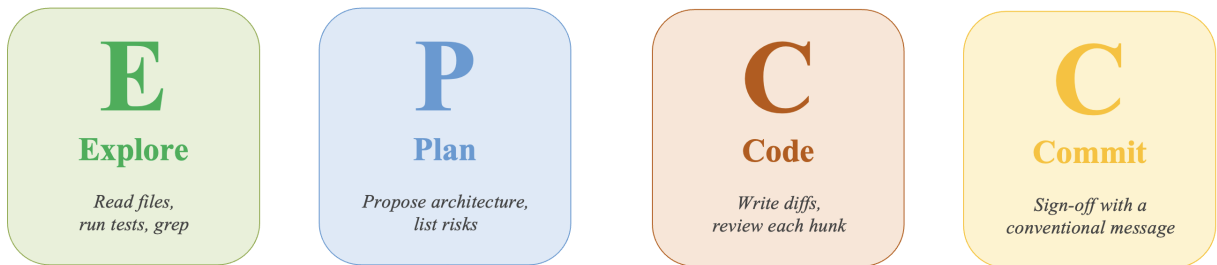
V. Workflow

Explore → Plan → Code → Commit

Trong phần này chúng ta tìm hiểu một *quy trình* – một sequence các bước được thiết kế để vibe coding tạo ra diff có chất lượng cao và an toàn cho mã nguồn đang xây dựng. Phần này giới thiệu quy trình đó dưới tên gọi **E→P→C→C**: Explore → Plan → Code → Commit. Đây là workflow chuẩn cho mọi commit cycle, từ task đơn giản đến refactor đa file phức tạp.

V.1. Tổng quan workflow E→P→C→C

Workflow chuẩn của vibe coding gồm bốn phase, được gọi tắt là E→P→C→C. Đây là quy trình của *một commit cycle* – từ lúc bạn nhận yêu cầu cho đến khi push code lên branch:



One commit cycle = four phases. Skip none.

Hình 12: Bốn phase của một commit cycle: Explore → Plan → Code → Commit.

Explore là khám phá tài nguyên hiện có, Plan là cân nhắc, lên kế hoạch (trade-off, risk), Code là thực thi (thận trọng, từng diff), Commit là sign-off (final review). Bỏ qua một phase nào cũng để lại lỗ hổng – ví dụ thiếu Plan dẫn đến code lan man, skip Explore dẫn đến chỉnh sửa code vi phạm tài nguyên sẵn có.

V.2. Phase 1 – Explore (Khám phá)

Phase đầu tiên có tên *Explore* bởi vì đây là giai đoạn trong vòng lặp mà Claude Code đưa ra những phân tích chuyên sâu về trạng thái đang có của mã nguồn hoặc các tài nguyên. Mục tiêu không phải là sửa gì cụ thể, mà là xây dựng một bức tranh chính xác về mã nguồn trước khi sửa. Trước khi đề xuất bất kỳ thay đổi nào, cần biết file nào liên quan, entry point ở đâu, quy định và quy ước hiện tại ra sao, và những giả định ngầm nào mà code base đang dựa vào. Bốn yếu tố then chốt của Explore:

- **Mục tiêu:** bức tranh chính xác về mã nguồn
- **Kỹ thuật phổ biến:**

- “Find files matching X” – dùng built-in tool Glob
- “Read main.py and summarize” – dùng Read + LLM summarize
- “grep -r TODO for tech debt” – dùng Grep
- **Output:** một danh sách gồm files, entry points, existing patterns
- **Anti-pattern:** skip Explore → các thay đổi sẽ bỏ sót tài liệu sẵn có hoặc làm hỏng test sẵn có

💡 Example prompt cho Explore

“Find every file that imports requests, summarize what each one does in 1 sentence. List entry points (anything in `if __name__ == '__main__':`).”

V.3. Phase 2 – Plan (Lập kế hoạch)

Sau khi đã hiểu rõ về thư mục, cấu trúc file trong Explore, chúng ta cần lập một kế hoạch cụ thể. Plan là phase mà developer phân tích và đánh giá với đề xuất của Claude trước khi viết một dòng code nào. Đây là lúc cân nhắc trade-off, đánh giá risk, và đảm bảo cả người và AI đồng thuận về *cái gì sẽ thay đổi* trước khi bất kỳ file nào bị sửa. Để vào Plan Mode, bấm **Shift** + **Tab** hai lần – ở mode này Claude bị cấm sửa file (tool **Edit** bị disable), chỉ đề xuất bằng text. Đây là rào chắn kỹ thuật để đảm bảo chúng ta thực sự dừng lại đọc plan trước khi gõ tay.

- **Trigger:** **Shift** + **Tab** × 2 – vào Plan Mode
- **Yêu cầu plan:** “Propose a plan to add feature X. Plan only – no code yet.”
- **Một plan tốt phải liệt kê:**
 - Files sẽ tạo/chỉnh sửa
 - Test cần thêm
 - Risk & trade-off
- **Review plan như review PR description:** approve, edit, hoặc yêu cầu phương án khác

💡 Example prompt cho Plan

“Propose a plan to add JWT authentication to the `/login` endpoint. Plan only – no code yet. List files to modify, tests to add, and one trade-off (e.g. refresh token strategy).”

V.4. Phase 3 – Code (Sinh diff)

Có plan, sau đó chuyển sang giai đoạn thực thi. Phase Code là phase mà code thực sự được sinh ra – tuy nhiên, cần nhấn mạnh: người dùng không gõ code mà tập trung duyệt code. Sau khi plan được approved, thoát Plan Mode (**Shift** + **Tab**) để trở lại Approval Mode (mặc định). Trong mode này, Claude đề xuất diff theo từng file, và chúng ta review từng *hunk* (đoạn thay đổi liên tục) trước khi accept. Khác với code truyền thống, ở đây developer là một code reviewer chuyên nghiệp – nhiệm vụ là phát hiện sai sót, không phải tạo ra logic mới.

- **Mode:** Approval Mode (mặc định khi mới mở Claude Code)
- **Cách review từng hunk:**
 - Logic có đúng không?
 - Có khớp hướng dẫn trong CLAUDE.md không?
 - Có thêm dependency nào không?
- **Approve** → file được cập nhật
- **Chạy test inline:** `!pytest tests/` hoặc `!npm test`
- **Iterate** ngay trong session nếu test fail

💡 Example prompt cho Code

“Implement plan step 1 first. Stop before step 2 so I can review the diff.”

V.5. Phase 4 – Commit (Sign-off)

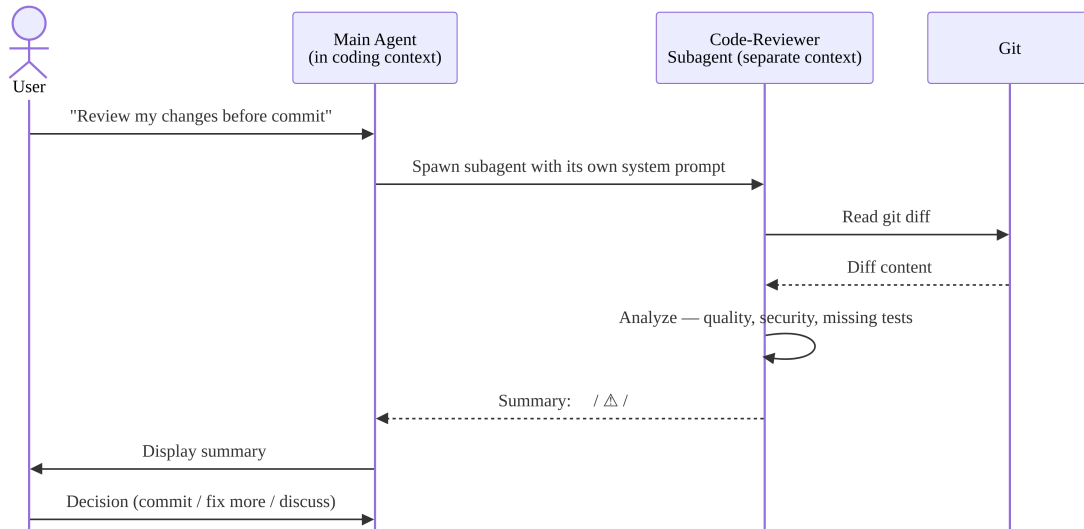
Phase cuối trong chuỗi commit cycle. Cho dù diff đã chỉnh sửa rất kỹ ở Phase 3, vẫn còn một bước cuối cùng – nhìn lại *toàn bộ* thay đổi, soạn commit message phản ánh sự thay đổi, và nhấn Enter. Đây là điểm con người (không phải Claude) xác nhận những thay đổi; pattern thiết kế của Claude Code luôn giữ git commit là checkpoint của developer, không phải của AI.

- **Final review:** `!git diff` để xem toàn bộ thay đổi
- **Conventional commit format:**
 - `feat(scope): summary`
 - `fix(scope): summary`
 - `refactor: ... / docs: ... / test: ...`
- **Claude có thể draft message:** “Suggest a commit message for these changes”
- **Chúng ta nhấn Enter – không bao giờ auto-commit:** đây là checkpoint của con người

💡 Example prompt cho Commit

“Suggest a conventional commit message for the changes since HEAD~1. Format: feat(scope): summary.”

Một workflow nâng cao là gọi *code-reviewer subagent* trước khi commit. Subagent này có context riêng (không bị ảnh hưởng bởi main agent đã code lâu), nó đọc `git diff` và trả về phản hồi ngắn gọn. Sơ đồ tương tác như sau:



Hình 13: Quy trình code-reviewer subagent. Subagent này sẽ có context riêng để tăng tính phản biện công bằng.

Subagent thường gán nhãn được những thứ main agent bị thiếu: test thiếu cho edge case, security concern (hardcoded secret, SQL injection tiềm ẩn), code smell (function quá dài, naming không rõ). Sau khi nhận summary, bạn quyết định: commit luôn, fix thêm, hay thảo luận thêm với Claude. Phần triển khai subagent **agent-reviewer** cụ thể sẽ được đề cập ở phần sau.

V.6. Quản lý bộ nhớ với CLAUDE.md – Memory

Vấn đề? Sau khi đã hiểu workflow $E \rightarrow P \rightarrow C \rightarrow C$, có một bài toán thực tế đặt ra: mỗi session mới, Claude lại bắt đầu từ con số 0 – không biết stack tech của chúng ta (Python? Node? TypeScript?), không biết convention coding bạn theo (PEP 8? Black? Prettier?), không biết project là web app, library, hay CLI, không biết những pet peeves cá nhân (“không bao giờ dùng `var`”, “luôn có type hints”). Hậu quả: chúng ta phải nhắc lại convention mỗi session, hoặc tệ hơn – chấp nhận diff sai convention.

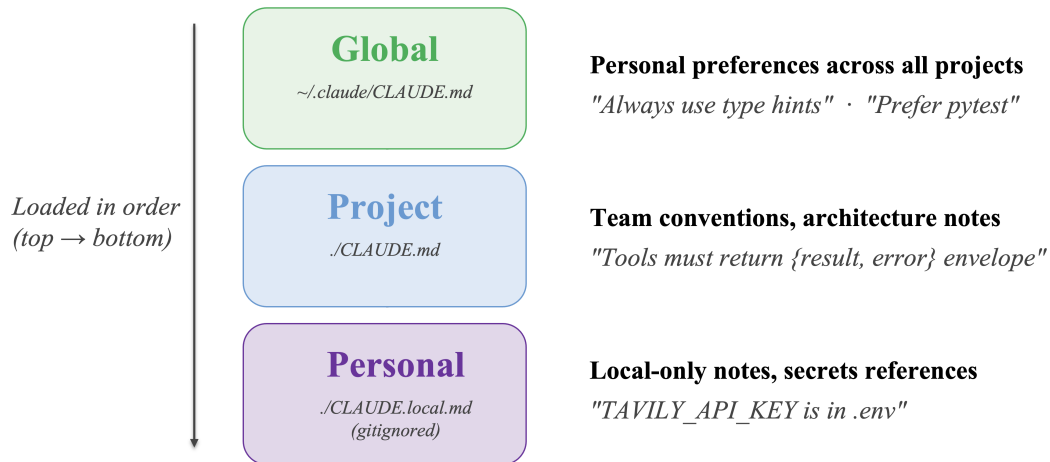
Giải pháp là CLAUDE.md – một file Markdown mà Claude Code đọc và chèn vào system prompt mỗi khi mở REPL. Để Claude Code hiểu *conventions của bạn* mà không cần nhắc lại mỗi session, dùng file CLAUDE.md. Hệ thống gồm ba cấp, được load theo thứ tự từ rộng đến hẹp:

- **Global** – `~/ .claude/CLAUDE.md`: preferences cá nhân áp dụng cho mọi project. Vd: “Always use type hints”, “Prefer pytest”.
- **Project** – `./CLAUDE.md`: convention team, kiến trúc, design decision của project hiện tại. Vd: “Tools must return `{result, error}` envelope”.
- **Personal** – `./CLAUDE.local.md` (gitignore): note cá nhân cho project, reference đến secret, todo list. Vd: “TAVILY_API_KEY is in `.env`”.

File load sau đè lên file load trước – nếu Project nói “no type hints” thì Global “always use type hints” bị ghi đè cho project này.

Lưu ý

File `./CLAUDE.local.md` không bao giờ được commit vào git. Để chắc chắn, thêm dòng `CLAUDE.local.md` vào `.gitignore` của project. Đây là nơi bạn ghi chú các tham chiếu hoặc thông tin của cá nhân, đường dẫn local, hoặc note cá nhân chưa chia sẻ với team.



Hình 14: Hệ thống CLAUDE.md ba cấp. Claude Code đọc và merge cả ba mỗi session. Thứ tự ưu tiên: Personal > Project > Global – khi có rule trùng.

V.7. @file references và !shell inline

Trong phần này chúng ta tìm hiểu về hai cú pháp có thể giúp mọi prompt Claude Code thành mini-IDE thật sự:

- **@file**: nạp nội dung file vào context một cách rõ ràng. Vd: `@main.py @tools.py` nạp cả hai file.
- **!command**: chạy shell command, output được thu thập vào context. Vd: `!pytest tests/ -v` chạy test và inject output.

3 ví dụ về cú pháp sử dụng @file và !command

```

1 > @main.py @tools.py
2   Compare these two files. Where do they share types?
3
4 > !pytest tests/test_tools.py -v
5   Tests just ran. Tell me which failed and why.
6
7 > @CLAUDE.md
8   Does my new function violate any convention listed here?
```

VI. Lab 1 – Build A ReAct Agent using Claude Code

Trong phần này, chúng ta áp dụng những nội dung quy trình trên vào thực hành. Lab 1 xây dựng một ReAct Agent từ đầu, sử dụng OpenAI function calling.

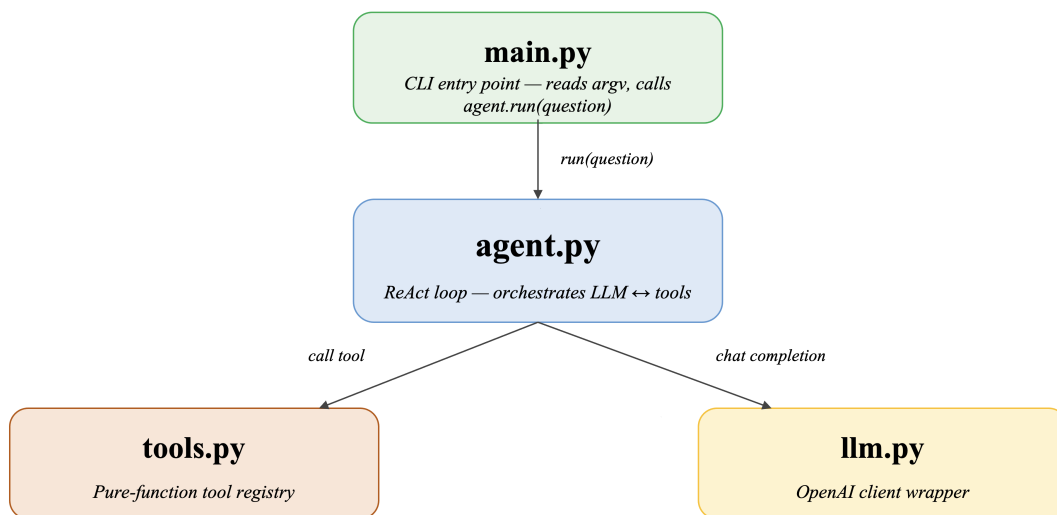
VI.1. Mục tiêu Lab 1

Trong Lab 1, chúng ta xây dựng từ đầu một ReAct Agent dùng OpenAI function calling. Agent này có thể trả lời câu hỏi nhiều bước qua vòng lặp ReAct (*Reasoning + Acting*). Lab này nhằm hai mục đích sau:

- **Học cấu trúc agent:** vòng lặp Thought → Action → Observation → Loop diễn ra thế nào ở mức code.
- **Trải nghiệm vibe coding:** chúng ta không viết một dòng nào, Claude Code viết toàn bộ ~250 dòng – chúng ta chỉ đặc tả và duyệt.

VI.2. Kiến trúc ReAct Agent

ReAct Agent gồm bốn module, mỗi module một nhiệm vụ rõ ràng:



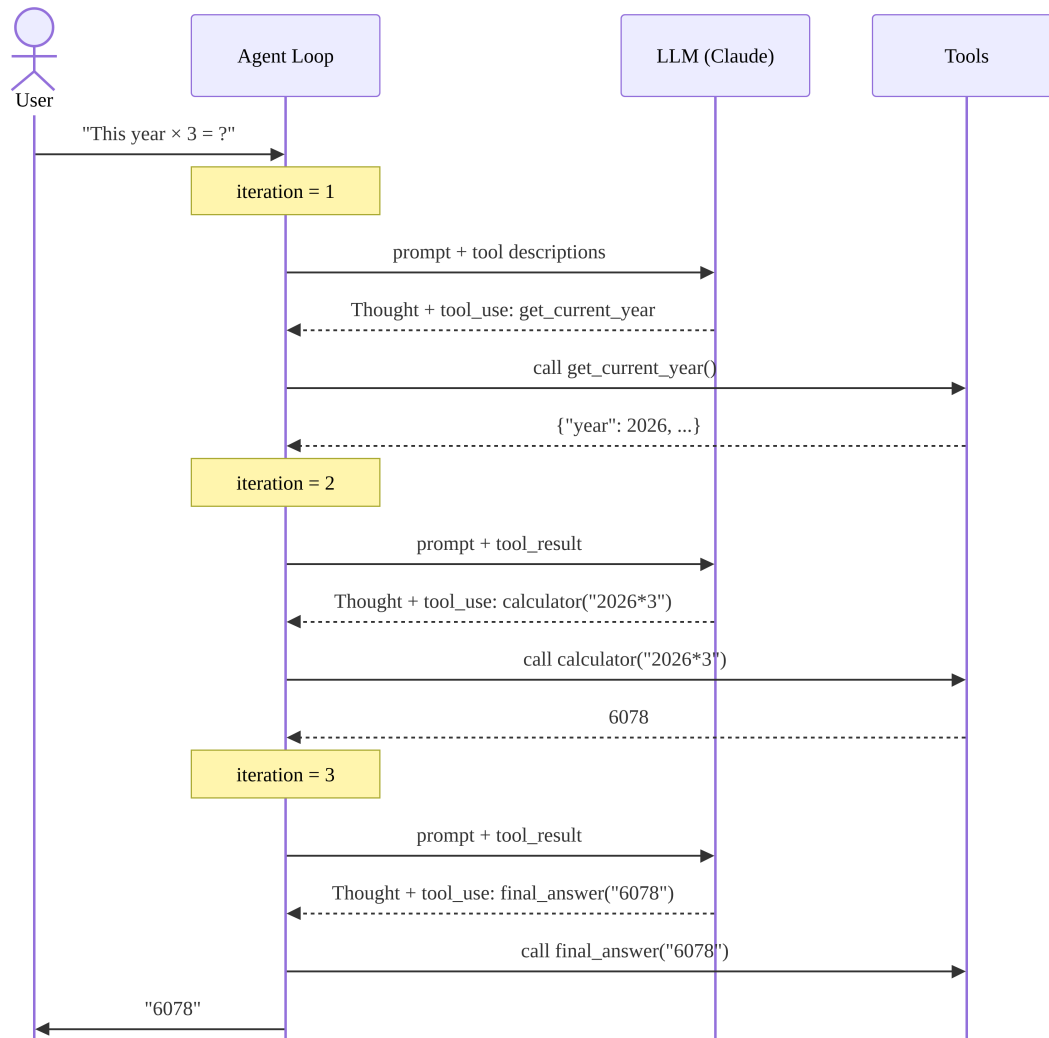
Hình 15: Kiến trúc ReAct Agent – bốn module, một ReAct loop.

Mục đích của thiết kế bốn module này là thiết kế theo chức năng, với mỗi file đảm nhiệm một chức năng nhất định:

- **main.py** – CLI entry point, nhận câu hỏi từ `argv`, gọi `agent.run(question)`.
- **agent.py** – ReAct loop, điều phối giữa LLM client và tools.

- **tools.py** – tool registry, chứa các function thuần Python (calculator, get_current_year, web_search mock, final_answer).
- **llm.py** – wrapper quanh OpenAI client cho gọn, dễ thay backend khác sau này.

Để hình dung bốn module này tương tác ra sao khi xử lý một câu hỏi cụ thể, hãy xem sequence diagram dưới đây. Chú ý cách mỗi iteration tạo thành một mini-cycle hoàn chỉnh – gửi context, nhận tool call, thực thi, và đưa kết quả vào context cho iteration tiếp theo:



Hình 16: Quy trình thực tế của một câu hỏi qua ReAct loop. Mỗi iteration: agent gửi context cho LLM, LLM đề xuất tool call, agent thực thi, đưa kết quả về cho LLM ở iteration tiếp. Khi LLM gọi `final_answer`, loop dừng.

LLM client chúng ta dùng là OpenAI GPT-4o. Bốn tool ban đầu:

- `calculator(expression)` – evaluate biểu thức số học thuần
- `get_current_year()` – trả về 2026 (kiểu int)

- `web_search(query)` – mock trong Lab 1 (sẽ thay bằng Tavily ở Lab 2)
- `final_answer(answer)` – tool đặc biệt báo hiệu kết thúc loop

VI.3. Vibe coding Lab 1 sử dụng Claude Code

Phần này xây dựng ReAct Agent từ đầu — từ workspace trống đến agent chạy được ba smoke test. Quy trình gồm chín bước, mỗi bước bám sát workflow $E \rightarrow P \rightarrow C \rightarrow C$ đã trình bày ở Mục V: bước 0 chuẩn bị môi trường + `CLAUDE.md`; bước 1 vào Plan Mode đặc tả kiến trúc; bước 2 review plan; các bước 3–7 sinh từng file và review từng diff; bước 8 smoke test cùng với pattern xử lý lỗi; bước 9 commit nhỏ theo phase. Phần dưới đây trình bày tuần tự.

Bước 0 – Chuẩn bị workspace và `CLAUDE.md`

Tạo folder dự án và file `CLAUDE.md` *trước khi mở Claude Code*. Một `CLAUDE.md` tốt từ đầu sẽ giúp chúng ta tiết kiệm nhiều giờ debug sau này.

Terminal — chuẩn bị môi trường

```
1 mkdir -p ~/aio-vibe-coding/mini-agent && cd ~/aio-vibe-coding/mini-agent
2 git init -q
3 python -m venv .venv && source .venv/bin/activate
4 pip install -q openai python-dotenv pytest
5 echo "OPENAI_API_KEY=sk-proj-xxxxxxxxxxxx" > .env
6 echo ".venv/" > .gitignore && echo ".env" >> .gitignore
7 touch CLAUDE.md
```

Sau đó mở `CLAUDE.md` bằng editor bất kỳ, ghi convention cho project:

CLAUDE.md – project memory ban đầu

```

1  # ReAct Agent  convention dự án
2
3  ## Stack
4  - Python 3.11, OpenAI SDK (>=1.0), python-dotenv, pytest
5  - Không dùng async; tất cả I/O đồng bộ cho dễ đọc.
6
7  ## Coding style
8  - Type hints trên mọi public function.
9  - Docstring (Google style) cho function/class public.
10 - Mỗi tool function trả về envelope: {"result": str, "error": str | None}.
11 - Không print() trong logic; dùng logging hoặc trả về string.
12
13 ## Architecture decisions
14 - 4 module: main.py, agent.py, tools.py, llm.py.
15 - ReAct loop có MAX_ITERATIONS = 10 (giới hạn cứng).
16 - LLM client (llm.py) bao bọc OpenAI để dễ thay backend sau (Anthropic, Mistral...).
17
18 ## Test
19 - pytest tests/ cho mọi module có logic.
20 - Mock OpenAI/Tavily ở tests/, không gọi network thật.
21
22 ## Plan-first policy
23 - Mọi tính năng >1 file -> BẮT BUỘC Plan Mode trước.

```

💡 Vì sao bước này quan trọng

Không có CLAUDE.md, Claude phải đoán convention → diff đầu tiên rất có thể dùng `async/await` hoặc `print()` debug, sau đó ta phải refactor lại. Việc viết CLAUDE.md tiết kiệm ít nhất thời gian sửa diff lệch convention.

Bước 1 – Mở REPL, vào Plan Mode, đặc tả yêu cầu

Trong folder `mini-agent`, gõ `claude` để mở REPL. Bấm `Shift` + `Tab` hai lần để vào Plan Mode (status bar dưới chuyển thành “Plan Mode – read-only”). Đây là lúc quan trọng nhất của cả Lab: prompt đầu tiên định hình toàn bộ kiến trúc mà Claude sẽ đề xuất.

Prompt 1 – Yêu cầu kiến trúc tổng thể (trong Plan Mode)

```
1 @CLAUDE.md
2
3 Tôi muốn build một ReAct Agent đơn giản. Đây là dự án học tập
4 cho khoá AIO -- mục đích là cho học viên thấy vòng lặp ReAct hoạt
5 động thế nào ở mức code.
6
7 Yêu cầu chức năng:
8 - Agent nhận một câu hỏi qua argv, trả về câu trả lời cuối.
9 - Có 4 tool: calculator(expression), get_current_year() trả về 2026,
10 web_search(query) MOCK trả về string giả ở Lab 1, và final_answer(answer).
11 - Dùng OpenAI Function Calling protocol chuẩn (model gpt-4o-mini).
12 - Agent loop có MAX_ITERATIONS = 10, raise nếu vượt.
13
14 Yêu cầu kiến trúc (theo CLAUDE.md):
15 - 4 module rõ ràng: main.py, agent.py, tools.py, llm.py.
16 - Separation of concerns: tools.py không biết gì về LLM,
17 llm.py không biết gì về tools, agent.py điều phối.
18
19 Đề xuất plan chi tiết:
20 - Mỗi module: tên function/class public, ngăn gọn responsibility.
21 - Thứ tự triển khai (file nào trước, vì sao).
22 - Test scaffold tối thiểu (file nào cần unit test, mock gì).
23 - Risks/trade-offs.
24
25 Plan only, không code.
```

Sau khoảng 20–30 giây, Claude trả về plan kiểu như sau (rút gọn để minh họa):

Plan output Claude trả về

```

1 PLAN: Build ReAct Agent (Lab 1)
2
3 # 1. Files affected (sẽ tạo mới chưa có file nào)
4 - tools.py      : 4 tool functions + TOOL_REGISTRY + TOOL_SCHEMAS
5 - llm.py        : OpenAIClient wrapper với .chat() trả response object
6 - agent.py      : ReActAgent class với .run(question) -> str
7 - main.py       : CLI entry point đọc argv, gọi agent.run
8 - tests/test_tools.py : pytest cho calculator, get_current_year
9
10 # 2. Implementation order (vì sao theo thứ tự này)
11 1. tools.py      không phụ thuộc gì, unit-testable ngay.
12 2. llm.py        chỉ phụ thuộc openai package, wrap chat.completions.
13 3. agent.py      phụ thuộc tools.py + llm.py.
14 4. main.py       phụ thuộc agent.py, đơn giản nhất, làm sau cùng.
15 5. tests/        viết song song với tools.py, mock OpenAI ở test agent.
16
17 # 3. Module responsibilities
18 - tools.py       : các tool dạng function thuần, không I/O ngoài. Calculator dùng
19                   ast.parse với whitelist (KHÔNG eval/exec).
20 - llm.py         : init OpenAI(), method .chat(messages, tools)
21                   -> response. Đọc API key từ os.getenv.
22 - agent.py       : class ReActAgent(client). Method .run(question) chứa
23                   vòng lặp for iteration in range(MAX_ITERATIONS).
24 - main.py        : sys.argv[1] -> agent.run -> print final.
25
26 # 4. Tool schemas (OpenAI function format)
27 - calculator(expression: str) -> {"result": str, "error": str|None}
28 - get_current_year() -> {"result": "2026", "error": None}
29 - web_search(query: str) -> mock string ở Lab 1
30 - final_answer(answer: str) -> terminate loop với answer
31
32 # 5. Risks & trade-offs
33 - Calculator dùng eval() đơn giản nhưng KHÔNG SAFE -> dùng ast.parse
34   + whitelist (chậm hơn ~10us nhưng an toàn). Theo CLAUDE.md.
35 - MAX_ITERATIONS = 10 đủ cho 3 smoke test, có thể chỉnh nếu cần.
36 - Mock web_search ở Lab 1 -> Lab 2 sẽ thay bằng Tavily REST.
37 - Không async để code đọc thẳng -> đánh đổi: không scale concurrent.
38
39 # 6. Success criteria
40 - pytest tests/ pass.
41 - python main.py "1 + 1 = ?" trả "1 + 1 = 2" trong <=2 iteration.
42 - python main.py "Năm nay nhân 3?" trả 6078 trong 2 iteration.
43 - python main.py "Thủ đô VN?" trả "Hà Nội" trong 1 iteration (zero-tool).
44
45 Approve plan? (Reply approve / edit / discuss)

```

Bước 2 – Review plan như review một PR description

Đây là nơi developer thật sự cần giám sát kỹ trong vibe coding. Đọc plan ở trên, chúng ta cần kiểm tra ba thứ: (1) kiến trúc có hợp lý không, (2) thứ tự triển khai có đúng không, (3) tiêu chí thành công có đo lường được không. Trong walkthrough này, plan trông tốt – một điểm muốn

chú ý thêm: yêu cầu Claude *dừng sau mỗi file* thay vì sinh hết một lần, để có thể review từng diff trước khi sang file kế.

Phản hồi review plan

```
1 Approve. Bổ sung 2 điều kiện thực thi:
2
3 1. Sau khi xong tools.py, DỪNG. Tôi review trước khi sang llm.py.
4 2. Đoạn calculator dùng ast.parse hãy whitelist chỉ
5   {Expression, BinOp, UnaryOp, Constant, Add, Sub, Mult, Div,
6     Pow, USub, FloorDiv, Mod}. Không cho Name, Call, Attribute.
7
8 Bắt đầu với tools.py.
```

Bước 3 – Sinh tools.py (file đầu tiên)

Bấm `Shift` + `Tab` thoát Plan Mode (status đổi thành “Approval mode”). Claude bắt đầu sinh diff cho `tools.py`. Trong Approval Mode, mỗi tool call (vd `Write tools.py`) sẽ xin xác nhận về bản preview của file, có thể bấm `y` accept, `n` reject, hoặc `a` accept + “don’t ask again for Write”.

REPL output – diff cho tools.py

```

1 Claude: Tôi sẽ tạo tools.py với 4 tool functions, ast-based
2 calculator whitelist, và TOOL_SCHEMAS theo OpenAI format.
3
4 [Tool: Write]
5 [File: tools.py]
6 [Diff preview 95 lines]
7
8 + """Pure-function tool registry for ReAct Agent (Lab 1)."""
9 + import ast
10 + from typing import Callable
11 +
12 + _ALLOWED_AST_NODES = {
13 +     ast.Expression, ast.BinOp, ast.UnaryOp, ast.Constant,
14 +     ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow,
15 +     ast.USub, ast.FloorDiv, ast.Mod,
16 + }
17 +
18 + def _is_safe(node: ast.AST) -> bool:
19 +     return all(type(n) in _ALLOWED_AST_NODES
20 +                for n in ast.walk(node))
21 +
22 + def calculator(expression: str) -> dict:
23 +     """Evaluate a pure arithmetic expression.
24 +     ... (docstring + body)
25 +     """
26 + ... (3 tool khác + TOOL_REGISTRY dict + TOOL_SCHEMAS list)
27
28 Approve? [y/n/a]:

```

Một số điểm chú ý từ code trên:

- Calculator dùng `ast.parse` + whitelist (đúng yêu cầu, KHÔNG eval).
- Mọi tool trả envelope `{result, error}` (đúng convention CLAUDE.md).
- Docstring có format input rõ (vd *"expression must be pure arithmetic, no variables"*).
- Type hints đầy đủ.

Cả bốn đều đạt → bấm ☐ y. File `tools.py` được tạo ra.

 Lưu ý

Đây là kết quả của quá trình cho ta thấy vibe coding hoạt động: 30 giây trước workspace còn trống, giờ đã có một file 95 dòng đúng convention, ta chỉ tốn vài giây review. Mỗi diff vẫn cần đọc qua – đây là cách quan trọng trong phân biệt vibe coding chuyên nghiệp với chế độ AI tự động hoá.

Bước 4 – Unit test tools.py

Trước khi sang llm.py, chạy test cho tools.py. Claude đã đề cập trong plan rằng sẽ viết tests/test_tools.py song song:

REPL — kiểm tra tools

```

1 > Viết tests/test_tools.py cover calculator (3 case: bình thường,
2   malformed, security pattern như __import__) và get_current_year.
3   Sau đó chạy thử.
4
5 Claude: [Tạo tests/test_tools.py với 5 test functions]
6         [Run: !pytest tests/ -v]
7
8 tests/test_tools.py::test_calculator_simple          PASSED
9 tests/test_tools.py::test_calculator_complex         PASSED
10 tests/test_tools.py::test_calculator_malformed      PASSED
11 tests/test_tools.py::test_calculator_blocks_attack  PASSED
12 tests/test_tools.py::test_get_current_year           PASSED
13
14 5 passed in 0.08s

```

Năm test pass trong dưới 0.1 giây — tools.py đã sẵn sàng. Test xanh cũng cung cấp một mạng an toàn cho các thay đổi sau: khi sang Lab 2 refactor web_search từ mock sang Tavily REST API, cùng bộ test này sẽ phát hiện ngay nếu refactor làm break envelope {result, error}. Chuyển sang module tiếp theo.

Bước 5 – Sinh llm.py (wrapper OpenAI)

Tiếp tục trong cùng session (context vẫn còn nguyên):

Prompt sinh llm.py

```

1 Bây giờ tạo llm.py theo plan. Yêu cầu:
2 - Class OpenAIClient(model: str = "gpt-4o-mini")
3 - Method .chat(messages: list, tools: list | None) -> response object
4 - Đọc OPENAI_API_KEY qua dotenv (load_dotenv ở __init__)
5 - Không retry/timeout phức tạp ở Lab 1 (giữ đơn giản)

```

Claude sinh llm.py. Vì file này ngắn và chỉ gói đơn giản OpenAI client (__init__ đọc OPENAI_API_KEY qua dotenv, method chat() là proxy thuần), review chủ yếu kiểm tra ba điểm: type hints đầy đủ, load_dotenv() chạy trong constructor, và không có retry/timeout phức tạp (phù hợp scope Lab 1). Accept.

Bước 6 – Sinh agent.py

Chúng ta sử dụng prompt như sau.

Prompt sinh agent.py

```

1 Tạo agent.py với class ReActAgent. Yêu cầu chi tiết:
2
3 1. Constructor: __init__(self, client: OpenAIClient).
4 2. Method run(question: str) -> str.
5 3. Trong run:
6     - Khởi tạo messages = [system, user].
7     - SYSTEM_PROMPT giải thích role + danh sách tool có sẵn.
8     - for iteration in range(MAX_ITERATIONS = 10):
9         a) Gọi self.client.chat(messages, TOOL_SCHEMAS).
10        b) NGAY LẬP TỨC append response.message vào messages
11           (đây là Lỗi 1 trong CLAUDE.md anti-patterns: dễ quên).
12        c) Nếu finish_reason == "stop" -> return msg.content.
13        d) Nếu tool_calls: với mỗi call, json.loads(arguments),
14           gọi TOOL_REGISTRY[call.function.name](**args),
15           append role="tool" với tool_call_id=call.id và
16           content=json.dumps(result).
17           DỪNG call.id (KHÔNG tự đặt UUID Lỗi 2 trong anti-patterns).
18           DỪNG json.dumps cho content (KHÔNG str(dict) Lỗi 4).
19     - Sau loop: raise RuntimeError("Max iterations exceeded").
20
21 4. Log mỗi iteration: "[Iter k] Action: name(args)" + "Observation: ..."
22
23 Tổng độ dài kỳ vọng: ~40-60 dòng.

```

Claude sinh agent.py. Review thật kỹ ở 4 điểm:

- Có dòng messages.append(msg) ngay sau response không?
- tool_call_id=call.id dùng nguyên id từ response?
- content=json.dumps(result), không phải str(result)?
- Có for iteration in range(MAX_ITERATIONS) kèm raise cuối?

Cả bốn đều đạt → accept.

Bước 7 – Sinh main.py

Prompt sinh main.py

```

1 Cuối cùng tạo main.py. Đơn giản:
2 - Đọc sys.argv[1] làm question (raise nếu thiếu argv).
3 - Init OpenAIClient, ReActAgent.
4 - Gọi agent.run(question).
5 - print("Final Result:", result).

```

Claude sinh main.py 15 dòng. Accept. Toàn bộ project giờ có 5 file: tools.py, llm.py, agent.py, main.py, tests/test_tools.py. Tổng 250 dòng code – người dùng không cần gõ lần lượt các dòng code.

Bước 8 – Smoke test thật và iterate

Đây là moment kiểm thử. Chạy ba kịch bản đã đặc tả ở plan:

Terminal — chạy ba smoke test

```

1 python main.py "1 + 1 = ?"
2 [Iter 1] Action: calculator({'expression': '1 + 1'})
3 [Iter 1] Observation: 2
4 [Final] Answer: 1 + 1 = 2.
5 Final Result: 1 + 1 = 2.
6
7 python main.py "Năm nay nhân ba bằng bao nhiêu?"
8 [Iter 1] Action: get_current_year({})
9 [Iter 1] Observation: 2026
10 [Iter 2] Action: calculator({'expression': '2026 * 3'})
11 [Iter 2] Observation: 6078
12 [Final] Answer: Năm nay nhân ba bằng 6078.
13 Final Result: Năm nay nhân ba bằng 6078.
14
15 python main.py "Thủ đô của Việt Nam là gì?"
16 [Final] Answer: Thủ đô của Việt Nam là Hà Nội.
17 Final Result: Thủ đô của Việt Nam là Hà Nội.
```

Cả ba test pass ngay lần chạy đầu cho thấy kết quả tốt. Tuy nhiên trong thực tế, thường gặp một số lỗi khi chạy. Cách xử lý đúng kiểu vibe coding: paste log lỗi ngược lại cho Claude, để model tự diagnose và fix. Đây chính là phần “copy-paste” trong vòng lặp See→Say→Run→Copy-paste của Karpathy.

Pattern xử lý lỗi

```

1 > Test 1 bị infinite loop. Output đến iteration 10 đều là
2 cùng calculator("1 + 1"). Đoán lỗi ở chỗ messages.append.
3
4 Claude: [Read agent.py]
5         Đúng. Thiếu dòng append response.message sau khi nhận từ LLM.
6         Hiện tại chỉ append tool result, không append assistant message.
7
8         [Edit agent.py thêm 1 dòng]
9         + messages.append(msg)
10
11         Test lại chứ?
12
13 > Có, !python main.py "1 + 1 = ?"
14
15 [Iter 1] Action: calculator({'expression': '1 + 1'})
16 [Iter 1] Observation: 2
17 [Final] Answer: 1 + 1 = 2.
18
19 Pass.
```

Bước 9 – Commit từng phase

Sau mỗi smoke test pass, commit ngay để kiểm soát quá trình sinh và sửa code.

Git history

```
1 $ git log --oneline
2 abc1234 feat(agent): fix infinite loop append assistant message
3 def5678 feat(main): add CLI entry point
4 abc9876 feat(agent): add ReAct loop with MAX_ITERATIONS
5 def0011 feat(llm): add OpenAIClient wrapper
6 abc2233 feat(tools): add calculator + mock web_search + schemas
7 def4455 chore: init project (CLAUDE.md, .gitignore, venv)
```

Vibe coding workflow trong các bước trên là khuôn mẫu cho mọi project

Workflow vừa rồi – (0) setup + CLAUDE.md, (1) Plan Mode + prompt tổng, (2–7) generate từng file rồi review, (8) smoke test + iterate, (9) commit nhỏ – không chỉ áp dụng cho Lab 1. Ở Lab 2 (Mục VIII), khi nâng cấp Agent với Tavily/Subagent/Skill/MCP, ta sẽ thấy chính các bước này lặp lại thường xuyên. Vì vậy điểm mà chúng ta cần ghi nhớ: *kỹ năng vibe coding là kỹ năng vận hành workflow, không phải chỉ tập trung tối ưu viết prompt.*

Những nội dung tiếp theo trong chương này là các file code được sinh ra sau quá trình vibe coding với Claude Code ở các bước trên.

VI.4. Tool Registry – tools.py

Mỗi tool là một function Python thuần, kèm theo JSON schema theo chuẩn OpenAI. Schema này được truyền vào tham số `tools=` khi gọi `chat.completions.create`, giúp LLM biết tool nào có sẵn và mỗi tool cần tham số nào.

tools.py – tool registry đầy đủ với 4 tool + schema + registry

```

1  """Pure-function tool registry for ReAct Agent (Lab 1)."""
2  import ast
3  from typing import Any, Callable
4
5  # Whitelist các loại node AST cho calculator -- KHÔNG dùng eval thô.
6  _ALLOWED_AST_NODES = {
7      ast.Expression, ast.BinOp, ast.UnaryOp, ast.Constant,
8      ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow,
9      ast.USub, ast.FloorDiv, ast.Mod,
10 }
11
12
13 def _is_safe(node: ast.AST) -> bool:
14     """Kiểm tra mọi node trong AST đều nằm trong whitelist."""
15     return all(type(n) in _ALLOWED_AST_NODES for n in ast.walk(node))
16
17
18 def calculator(expression: str) -> dict:
19     """Evaluate một biểu thức số học thuần.
20
21     Args:
22         expression: chuỗi biểu thức (vd "2026 * 3"). Không hỗ trợ biến,
23                     không hỗ trợ gọi function.
24
25     Returns:
26         {"result": str, "error": str | None}
27     """
28     try:
29         node = ast.parse(expression, mode="eval")
30         if not _is_safe(node):
31             return {"result": "", "error": "Unsafe expression"}
32         value = eval(compile(node, "<calc>", "eval"))
33         return {"result": str(value), "error": None}
34     except Exception as e:
35         return {"result": "", "error": str(e)}
36
37
38 def get_current_year() -> dict:
39     """Trả về năm hiện tại (hard-code 2026 cho Lab 1)."""
40     return {"result": "2026", "error": None}
41
42
43 def web_search(query: str) -> dict:
44     """Mock web search cho Lab 1. Lab 2 sẽ thay bằng Tavily API thật."""
45     return {
46         "result": f"[MOCK] Top result for '{query}': Lorem ipsum content.",
47         "error": None,
48     }
49
50
51 def final_answer(answer: str) -> dict:
52     """Tool đặc biệt -- báo hiệu kết thúc vòng lặp ReAct."""
53     return {"final": True, "answer": answer}

```

tools.py – Schema theo chuẩn OpenAI Function Calling

```

1 # Registry -- map tên tool sang function thực thi.
2 TOOL_REGISTRY: dict[str, Callable[..., dict]] = {
3     "calculator": calculator,
4     "get_current_year": get_current_year,
5     "web_search": web_search,
6     "final_answer": final_answer,
7 }
8 # Schema theo chuẩn OpenAI Function Calling -- pass vào tham số tools=.
9 TOOL_SCHEMAS: list[dict[str, Any]] = [
10     {"type": "function", "function": {
11         "name": "calculator",
12         "description": "Evaluate pure arithmetic expression "
13             "(no variables, no function calls), e.g. '2026 * 3'.",
14         "parameters": {
15             "type": "object",
16             "properties": {"expression": {"type": "string"}},
17             "required": ["expression"],
18         },
19     }},
20     {"type": "function", "function": {
21         "name": "get_current_year",
22         "description": "Return the current year. No arguments.",
23         "parameters": {"type": "object", "properties": {}, "required": []},
24     }},
25     {"type": "function", "function": {
26         "name": "web_search",
27         "description": "Search the web. Returns top result as string. "
28             "(Lab 1: mocked; Lab 2: real Tavily API.)",
29         "parameters": {
30             "type": "object",
31             "properties": {"query": {"type": "string"}},
32             "required": ["query"],
33         },
34     }},
35     {"type": "function", "function": {
36         "name": "final_answer",
37         "description": "Submit the final answer and terminate the loop.",
38         "parameters": {
39             "type": "object",
40             "properties": {"answer": {"type": "string"}},
41             "required": ["answer"],
42         },
43     }},
44 ]

```

Có ba điểm thiết kế quan trọng trong file này. Thứ nhất, calculator dùng `ast.parse` với whitelist các node thay vì `eval()` thuần – đây là rào chắn bảo mật ngăn input độc hại như `__import__('os').system('rm -rf /')`. Thứ hai, mọi tool đều trả về envelope dạng `{result, error}` nhất quán; điều này giúp `agent.py` không cần biết logic nội bộ của từng tool – chỉ cần kiểm tra `error is None` để rẽ nhánh. Thứ ba, `final_answer` là một tool đặc biệt trả về cờ

`final: True` – agent loop sẽ dừng ngay khi nhận được cờ này, không cần đợi `finish_reason = "stop"`.

Vì sao tool result phải có envelope `{result, error}`

Có hai lý do. **Một**, LLM cần biết phân biệt thành công và thất bại để quyết định bước tiếp; nếu không, exception nuốt mất sẽ làm agent tin rằng tool đã trả về rỗng. **Hai**, format thống nhất giúp `agent.py` không cần biết logic nội bộ của từng tool – chỉ cần check `error is None` để rẽ nhánh.

VI.5. LLM Client – `llm.py`

Đây là file chứa mã nguồn tương tác với các mô hình.

`llm.py` – wrapper gọn quanh OpenAI client

```

1  """OpenAI client wrapper for ReAct Agent."""
2  import os
3
4  from dotenv import load_dotenv
5  from openai import OpenAI
6
7
8  class OpenAIClient:
9      """Lightweight wrapper quanh OpenAI client.
10
11      Tách lớp này ra để dễ thay backend khác (Anthropic, Mistral...)
12      sau này mà không phải sửa agent.py.
13      """
14
15      def __init__(self, model: str = "gpt-4o-mini") -> None:
16          load_dotenv()
17          api_key = os.getenv("OPENAI_API_KEY")
18          if not api_key:
19              raise RuntimeError("OPENAI_API_KEY missing in .env")
20          self.client = OpenAI(api_key=api_key)
21          self.model = model
22
23      def chat(self, messages: list, tools: list | None = None):
24          """Gửi messages + tool schemas. Trả về response object của OpenAI."""
25          return self.client.chat.completions.create(
26              model=self.model,
27              messages=messages,
28              tools=tools,
29          )

```

Ba điểm thiết kế đáng chú ý. Đầu tiên, `load_dotenv()` chạy trong `__init__` để đảm bảo biến môi trường được nạp trước khi gọi `os.getenv`; nếu không có dòng này, key có thể không tìm thấy khi chạy từ folder khác. Thứ hai, model mặc định là `gpt-4o-mini` – rẻ và đủ tốt cho Lab, có thể ghi đè khi khởi tạo. Cuối cùng, phương thức `chat()` chỉ là proxy thuần – không thêm logic retry, timeout, hay logging ở Lab 1 để giữ code đơn giản; Lab 2 có thể bổ sung khi cần.

VI.6. Agent Loop – agent.py

Đây là file chính của ReAct Agent – file chứa class `ReActAgent` với phương thức `run()` cài đặt vòng lặp ReAct.

agent.py – Prompts

```
1 """ReAct Agent -- orchestrate giữa LLM và tools."""
2 import json
3
4 from llm import OpenAIClient
5 from tools import TOOL_REGISTRY, TOOL_SCHEMAS
6
7 MAX_ITERATIONS = 10
8
9 SYSTEM_PROMPT = """You are a helpful ReAct agent. You have 4 tools:
10 - calculator(expression): evaluate arithmetic
11 - get_current_year(): get current year (2026)
12 - web_search(query): search the web (mocked in Lab 1)
13 - final_answer(answer): submit final answer and terminate
14
15 Reasoning steps:
16 1. Analyze the user's question.
17 2. Decide if a tool is needed. If yes, call it; if you already know
18   the answer (from training data), respond directly.
19 3. After receiving tool output, decide next action.
20 4. When ready, call final_answer(answer) to terminate.
21
22 Be concise and accurate. Use tools only when necessary."""
```

agent.py – class ReActAgent + ReAct loop đầy đủ

```

1 class ReActAgent:
2     """ReAct loop orchestrator -- điều phối LLM và tools."""
3     def __init__(self, client: OpenAIClient) -> None:
4         self.client = client
5
6     def run(self, question: str) -> str:
7         """Chạy vòng lặp ReAct cho đến khi có câu trả lời cuối.
8         Args:
9             question: câu hỏi từ user.
10        Returns:
11            Câu trả lời cuối dạng string.
12        Raises:
13            RuntimeError nếu vượt MAX_ITERATIONS.
14        """
15        messages: list = [
16            {"role": "system", "content": SYSTEM_PROMPT},
17            {"role": "user", "content": question},
18        ]
19        for iteration in range(MAX_ITERATIONS):
20            response = self.client.chat(messages, tools=TOOL_SCHEMAS)
21            choice = response.choices[0]
22            msg = choice.message
23            messages.append(msg)
24            # Trường hợp 1: LLM trả lời trực tiếp, không gọi tool.
25            if choice.finish_reason == "stop":
26                print(f"[Final] Answer: {msg.content}")
27                return msg.content
28            # Trường hợp 2: LLM gọi tool -- xử lý từng tool call.
29            for call in msg.tool_calls:
30                args = json.loads(call.function.arguments)
31                tool_fn = TOOL_REGISTRY[call.function.name]
32                result = tool_fn(**args)
33                print(f"[Iter {iteration+1}] Action: "
34                      f"{call.function.name}({args})")
35                print(f"[Iter {iteration+1}] Observation: {result}")
36                # Nếu tool là final_answer -> dừng ngay, không cần loop tiếp.
37                if result.get("final"):
38                    print(f"[Final] Answer: {result['answer']}")
39                    return result["answer"]
40                # CRITICAL: tool_call_id phải khớp với call.id từ LLM.
41                # CRITICAL: content phải là string (json.dumps), không phải dict.
42                messages.append({
43                    "role": "tool",
44                    "tool_call_id": call.id,
45                    "content": json.dumps(result),
46                })
47            raise RuntimeError(
48                f"Max iterations ({MAX_ITERATIONS}) exceeded -- "
49                "agent không hội tụ về final_answer."
50            )

```

Bốn điểm dễ sai cần kiểm tra mỗi khi đọc lại code này:

- Dòng `messages.append(msg)` sau khi nhận response từ LLM – nếu quên dòng này, LLM sẽ thấy tool result mà không thấy chính nó vừa đề xuất tool nào, dẫn đến infinite loop. Đây là Bug #1 phổ biến nhất.
- `tool_call_id` phải khớp đúng với `call_id` từ response LLM – không được tự đặt UUID mới, OpenAI sẽ trả về HTTP 400.
- `content` của tool message phải là string (qua `json.dumps`) – không được là dict thô, OpenAI yêu cầu nghiêm ngặt.
- Có `for iteration in range(MAX_ITERATIONS)` kèm `raise` cuối – đây là giới hạn an toàn ngăn agent đi mãi không dừng (vd khi system prompt thiếu hướng dẫn rõ ràng).

VI.7. CLI Entry Point – `main.py`

File tổng hợp, code chính điều phối main: đọc câu hỏi từ `sys.argv`, khởi tạo agent, gọi `run()`, in kết quả. Đây là điểm vào duy nhất khi chạy lệnh `python main.py "câu hỏi"`.

main.py – CLI entry point

```

1  """CLI entry point cho ReAct Agent."""
2  import sys
3
4  from agent import ReActAgent
5  from llm import OpenAIClient
6
7
8  def main() -> None:
9      if len(sys.argv) < 2:
10         print('Usage: python main.py "<question>"')
11         sys.exit(1)
12
13         question = sys.argv[1]
14         client = OpenAIClient()
15         agent = ReActAgent(client)
16         result = agent.run(question)
17         print(f"Final Result: {result}")
18
19
20 if __name__ == "__main__":
21     main()

```

VI.8. Unit Tests – `tests/test_tools.py`

Lab 1 không yêu cầu test toàn diện, nhưng việc viết test cho `tools.py` có hai lợi ích: (a) đảm bảo whitelist calculator hoạt động đúng với input độc hại, và (b) cung cấp ví dụ để Claude sinh test cho các file phức tạp hơn ở Lab 2.

tests/test_tools.py – 7 test cover 4 tool

```

1  """Unit tests for tools.py -- chạy bằng `pytest tests/`."""
2  import pytest
3  from tools import calculator, final_answer, get_current_year, web_search
4  # ----- calculator -----
5  def test_calculator_simple():
6      assert calculator("1 + 1") == {"result": "2", "error": None}
7
8  def test_calculator_complex():
9      assert calculator("2026 * 3") == {"result": "6078", "error": None}
10
11 def test_calculator_malformed():
12     """Biểu thức không hợp lệ phải trả error, không raise."""
13     result = calculator("1 + ")
14     assert result["error"] is not None
15     assert result["result"] == ""
16
17 def test_calculator_blocks_attack():
18     """Whitelist phải chặn được import/call ngoài số học."""
19     payload = "__import__('os').system('echo hacked')"
20     result = calculator(payload)
21     assert result["error"] is not None
22     assert result["result"] == ""
23
24 # ----- get_current_year -----
25 def test_get_current_year():
26     assert get_current_year() == {"result": "2026", "error": None}
27
28 # ----- web_search (mock) -----
29 def test_web_search_mock_returns_string():
30     result = web_search("Python language")
31     assert "MOCK" in result["result"]
32     assert "Python language" in result["result"]
33     assert result["error"] is None
34
35 # ----- final_answer -----
36 def test_final_answer_sets_flag():
37     result = final_answer("Hà Nội")
38     assert result == {"final": True, "answer": "Hà Nội"}

```

Bảy test ngắn này chạy trong dưới 0.1 giây và cover toàn bộ bốn tool. Test cho calculator đáng chú ý nhất – `test_calculator_blocks_attack` đảm bảo whitelist của ta không bị bypass bởi các payload kiểu inject code. Khi sang Lab 2, ta sẽ thêm test cho `web_search` qua mock của `httpx.post` để test offline.

Sau khi hoàn thiện đủ năm file (`tools.py`, `llm.py`, `agent.py`, `main.py`, `tests/test_tools.py`) tổng cộng khoảng 250 dòng code, ta chạy ba smoke test ở phần dưới để xác nhận hệ thống hoạt động đúng.

VI.9. Smoke Tests

Sau khi build xong, ta chạy ba smoke test theo độ phức tạp tăng dần:

Test #1 – “1 + 1 = ?” (Một tool call)

Output expected

```
1 $ python main.py "1 + 1 = ?"
2
3 [Iter 1] Action: calculator({'expression': '1 + 1'})
4 [Iter 1] Observation: 2
5
6 [Final] Answer: 1 + 1 = 2.
7 Final Result: 1 + 1 = 2.
```

Test #2 – “Năm nay nhân ba bằng bao nhiêu?” (Chuỗi hai tool)

Output expected

```
1 $ python main.py "Năm nay nhân ba bằng bao nhiêu?"
2
3 [Iter 1] Action: get_current_year({})
4 [Iter 1] Observation: 2026
5 [Iter 2] Action: calculator({'expression': '2026 * 3'})
6 [Iter 2] Observation: 6078
7
8 [Final] Answer: Năm nay nhân ba bằng 6078.
9 Final Result: Năm nay nhân ba bằng 6078.
```

Test #3 – “Thủ đô của Việt Nam là gì?” (Zero-tool answer)

Output expected

```
1 $ python main.py "Thủ đô của Việt Nam là gì?"
2
3 [Final] Answer: Thủ đô của Việt Nam là Hà Nội.
4 Final Result: Thủ đô của Việt Nam là Hà Nội.
```

Lưu ý

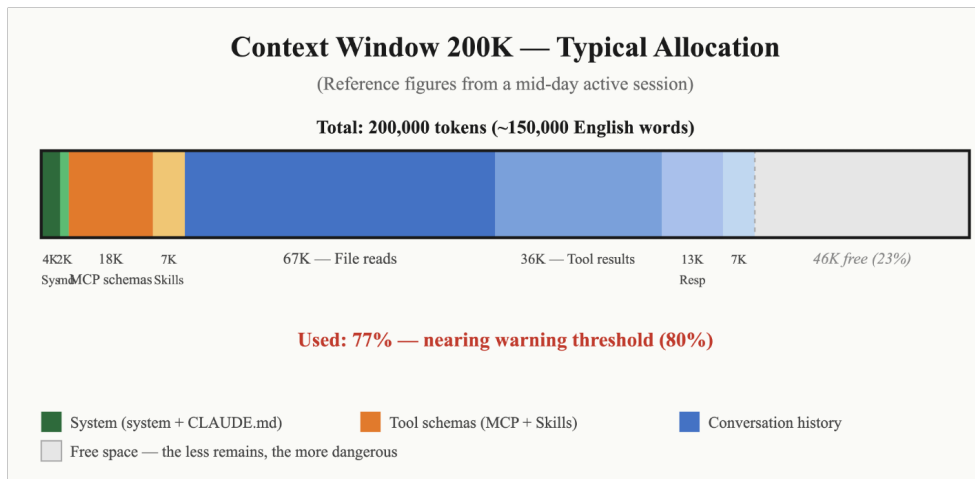
Test #3 cho thấy một đặc trưng quan trọng: *không phải mọi câu hỏi đều cần tool*. Khi LLM biết sẵn câu trả lời (từ training data), nó sẽ trả về `finish_reason = "stop"` ngay ở vòng đầu, không có `tool_calls`. Đây là hành vi đúng – agent loop phải handle case này gọn gàng, không assume mọi run đều có tool call.

VII. Context Engineering

Sau khi xây xong ReAct Agent, chúng ta đã tiếp xúc trực tiếp với một thực tế kỹ thuật quan trọng: mọi tương tác với LLM đều *tốn token*. Mỗi lần Read file, mỗi tool result, mỗi response của agent đều thêm dữ liệu vào context window – bộ nhớ làm việc của LLM. Khi context cạn, agent bắt đầu quên thông tin cũ, response chậm dần, và cuối cùng request có thể fail. Phần này dành cho *Context Engineering* – kỹ thuật quản lý context như một tài nguyên hữu hạn. Đây là kỹ năng sử dụng Claude Code rất quan trọng trong dự án thật, vì session dài hàng giờ luôn đặt ra bài toán context.

VII.1. Context Window 200K-1000K

Trước khi thảo luận về cách tối ưu, cần hiểu rõ context bao gồm những gì. Context window của Claude phụ thuộc vào phiên bản model: Claude 4.6 có cửa sổ 200,000 token (tương đương khoảng 150,000 từ tiếng Anh), trong khi Claude 4.7 mở rộng lên 1,000,000 token. Một con số nghe có vẻ lớn nhưng thực tế tiêu hao rất nhanh khi làm việc với mã nguồn thực. Đây là tài nguyên *hữu hạn* ở hai nghĩa: token đã dùng không lấy lại được trong cùng session, và mọi response của Claude đều ăn vào budget này (tổng input + output không vượt giới hạn). Trong phần phân tích dưới đây, chúng ta lấy trường hợp model 4.6 với 200,000 token làm ví dụ tham chiếu.



Hình 17: Anatomy của context window 200K – phân bổ token điển hình.

Phân bổ token điển hình:

- **System + tool schemas** (~5%): system prompt cố định + JSON schema của các tool.
- **CLAUDE.md** (~3%): nội dung ba file CLAUDE.md (Global + Project + Personal).
- **File reads** (~40%): tất cả file đã được Read qua tool, hoặc nạp qua `@file`.
- **Conversation history** (~40%): mọi user prompt + assistant response từ đầu session.
- **Output reservation** (~4%): khoảng trống dành cho response sắp tới.

- **Free** (~8%): phần còn lại.

Khi context vượt ~95%, Claude Code tự động trigger *auto-compact* (tóm tắt history). Nếu vượt 100%, request sẽ fail hoặc truncate.

VII.2. Lệnh `/context` và `/status`

Đo lường trước khi tối ưu – đây là nguyên tắc cơ bản. Hai lệnh quan trọng:

Trong Claude Code REPL

```
1 > /context
2 Context breakdown (137,250 / 200,000 tokens):
3   System + tools           8,420
4   CLAUDE.md (3 tiers)     4,180
5   Files (12 read)         68,300
6   Conversation             52,180
7   Output reservation      4,170
8
9 > /status
10 Session: 1h 23m sonnet-4-6 68.6% used
11 Breakdown:
12 System: 4K
13 CLAUDE.md: 2K
14 MCP tools: 18K (3 servers)
15   brave-search: 5K
16   filesystem: 4K
17   github: 9K
18 Skill descriptions: 7K
19 Conversation history: 103K
20 File reads: 67K
21 Tool results: 26K
22 Assistant responses: 10K
23 Free: 66K
```

Cách dùng thực tế: chạy `/status` mỗi 10 phút trong session dài. Khi thấy $\geq 80\%$, bắt đầu nghĩ đến `/compact`. `/context` dùng khi muốn biết *cái gì* đang chiếm budget nhiều nhất (thường là File reads).

VII.3. `/compact` vs `/clear` vs Auto-compact

Ba lệnh sau đây giúp giảm context – mỗi cái phù hợp với tình huống khác nhau:

Lệnh	/compact	/clear	auto-compact
Loại	Thủ công	Thủ công	Tự động (95%)
Hành vi	Tóm tắt history thành digest, giữ N turn gần nhất	Xoá toàn bộ history, làm lại từ đầu	Như /compact nhưng bị ép
Khi dùng	Debug session dài, muốn giữ context	Pivot sang task hoàn toàn mới	Không bao giờ – để hệ thống tự xử lý
Tốc độ	~5s	Tức thì	Tự kích hoạt

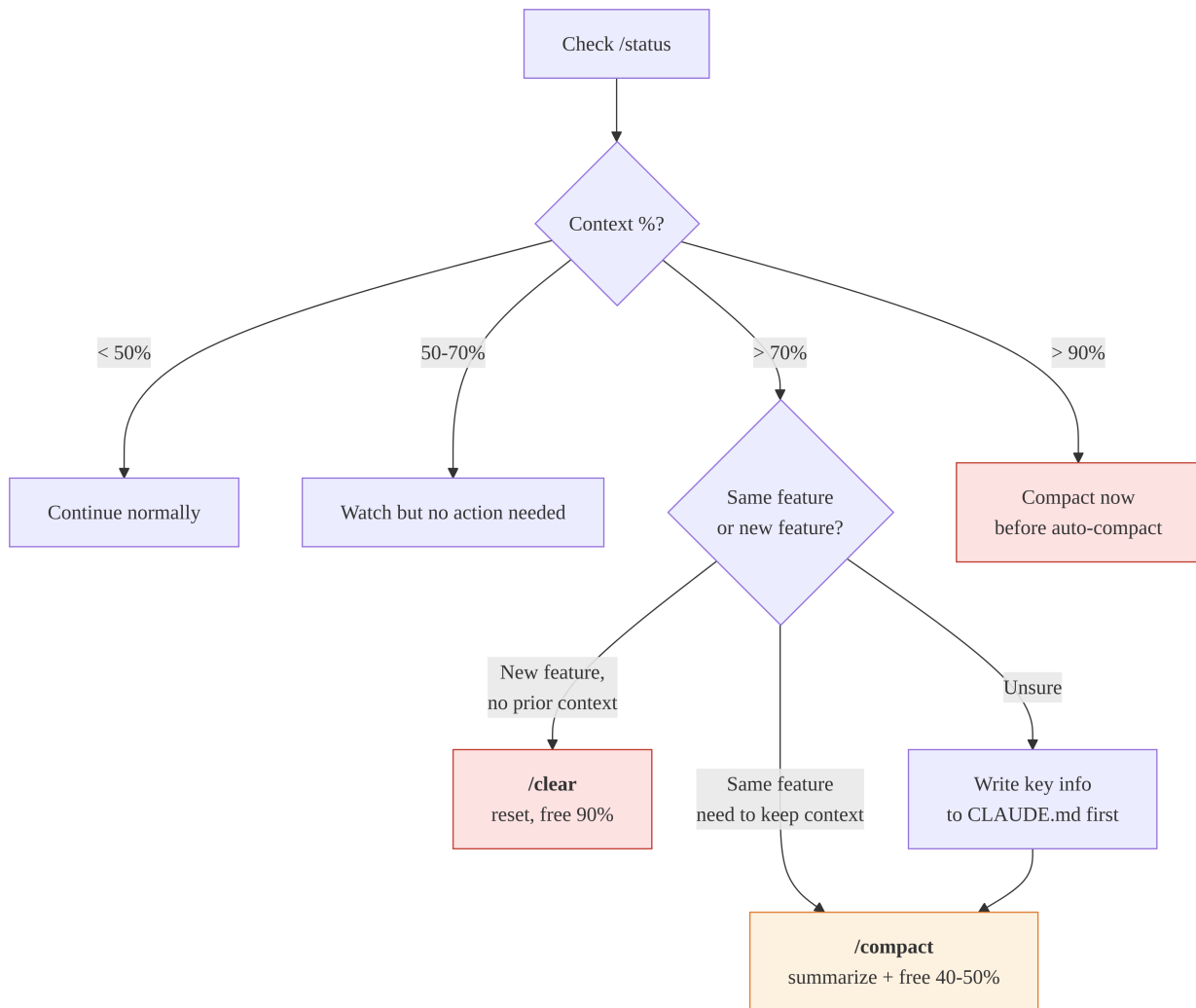
VII.4. Kiểm soát ngữ cảnh như thế nào?

Có ba chiến lược chính để giữ context window thoáng: (a) prompt cụ thể (tránh load cả file lớn khi chỉ cần một function), (b) tắt MCP server không cần thiết (mỗi server đăng ký schema vào budget mặc dù chưa dùng), (c) tận dụng để cho subagent thao tác (đẩy phần exploration nặng sang context con). Sơ đồ ra quyết định khi action đơn giản:

- Context > 80%?
 - **Không** → tiếp tục bình thường.
 - **Có** → Cùng task vẫn đang tiếp diễn?
 - * **Có** → /compact.
 - * **Không** → /clear.

Lưu ý

Thực tế rằng khi ngữ cảnh càng nhiều thì thông tin càng loãng nên khả năng sinh ra các kế hoạch hoặc thực thi kiểm tra các file các bị giảm đi. Vì vậy nên, việc thường xuyên kiểm tra context window rất quan trọng. Và con số lý tưởng cho các context window nên thường khoảng dưới 20-30% để giúp mô hình có thể hiểu biết về ngữ cảnh được tốt nhất. Tránh thiếu sót thông tin, đưa ra kế hoạch, thực thi, chỉnh sửa file không đúng.



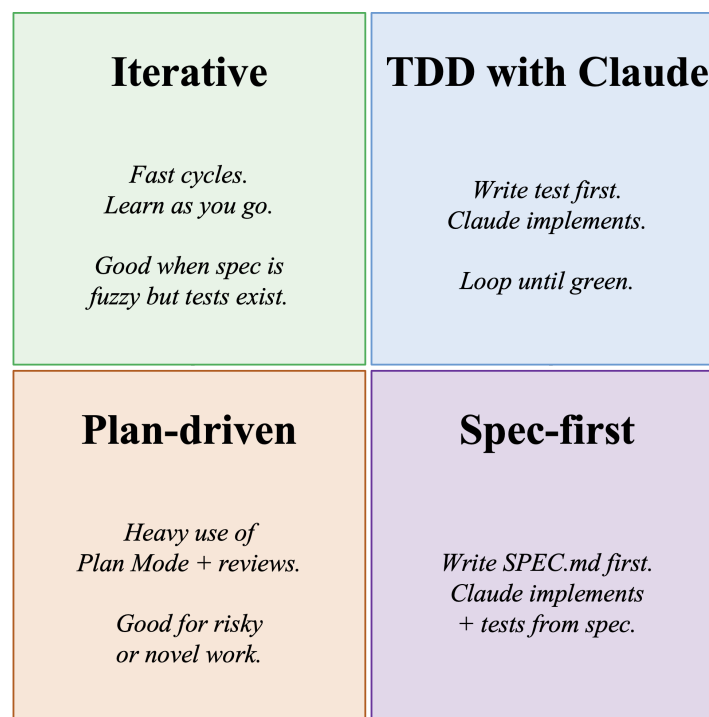
Hình 18: Cây quyết định khi check context. Trên 80% kiểm tra có còn cùng task hay không để chọn `/compact` hay `/clear`; vượt 95% là vùng auto-compact tự xử lý.

VII.5. Bốn workflow pattern nâng cao

Khi đã thành thạo workflow $E \rightarrow P \rightarrow C \rightarrow C$ cơ bản (Mục V), ta sẽ thấy nó là một khung tổng quát chứ chưa phải tất cả. Trong thực tế, từng dự án có đặc thù riêng và đòi hỏi cách áp dụng khung cơ bản theo những cách khác nhau. Có bốn variant nâng cao thường gặp, mỗi variant phù hợp với một tổ hợp khác nhau giữa độ rõ ràng của spec và độ phủ của test sẵn có:

Pattern	Khi nào dùng	Đặc điểm
Iterative	Spec mơ hồ, có test sẵn	Chu kỳ ngắn, học khi đi, refactor liên tục
TDD with Claude	Spec rõ + test bao phủ cao	Viết test trước, Claude triển khai, loop đến xanh
Plan-driven	Spec mơ hồ, không có test	Dùng Plan Mode nhiều, review kỹ từng diff
Spec-first	Spec rõ, ít test	Viết <code>SPEC.md</code> trước, Claude triển khai + tests từ spec

Bốn pattern này có thể hình dung trên một quadrant 2×2 theo hai chiều: độ rõ của spec (trục ngang) và độ phủ của test sẵn có (trục dọc). Mỗi góc quadrant tương ứng một pattern phù hợp:



Hình 19: Bốn pattern phân bố theo hai chiều: spec clarity (trục ngang) và test coverage (trục dọc). Chọn pattern phù hợp với góc quadrant ta đang đứng.

Phần dưới đây trình bày chi tiết từng pattern: bối cảnh áp dụng, workflow cụ thể, điểm cần chú ý, và ví dụ ngắn.

Pattern 1 – Iterative Refinement

Bối cảnh: spec từ stakeholder chưa rõ, nhưng test suite cũ vẫn chạy được. Đây là tình huống thường gặp khi ta nhận một mã nguồn cũ và task được mô tả mơ hồ kiểu “làm cho module X tốt hơn” mà chưa có định nghĩa rõ ràng tốt hơn nghĩa là gì.

Workflow điển hình – chu kỳ 5–15 phút mỗi vòng:

1. Chạy test hiện có để xác lập baseline (vd pass rate 85%, latency p95 = 120ms).

- Đưa cho Claude một prompt ngắn: “Cải thiện module X. Bắt đầu với một thay đổi nhỏ, đo lường trước khi thay đổi lớn.”
- Chạy lại test. Đo lường ba thứ: pass rate, performance, coverage.
- Đánh giá hướng đi: tốt hơn → tiếp tục cùng hướng; xấu đi → revert; không rõ → thảo luận với Claude trước khi tiếp.
- Commit nhỏ sau mỗi vòng cải thiện. Quay lại bước 2.

Đặc trưng: developer học khi đi – mỗi vòng iterate điều chỉnh hướng dựa trên signal cụ thể (test pass, benchmark) chứ không phải spec viết trước. Tránh được tình trạng viết spec hoàn hảo rồi mới biết hướng đó không khả thi.

Điểm cần lưu ý: Iterative dễ rơi vào tình trạng đi loanh quanh nếu không có tiêu chí dừng rõ ràng. Quy tắc thực dụng: sau 5 vòng iterate liên tiếp không cải thiện được số đo nào, ta phải dừng lại – xem xét lại approach hoặc chuyển sang Spec-first.

Ví dụ ngắn: tối ưu function `search()` của một search engine nội bộ. Test suite đo recall và latency. Vòng 1: thay đổi tokenizer → recall tăng 2%, latency tăng 5%. Trade-off chấp nhận được, commit. Vòng 2: thử cache layer → latency giảm 30% nhưng tốn memory 2GB. Discuss với Claude, quyết định: cache có giới hạn 500MB. Commit. Sau 6 vòng, kết quả cuối: recall +8%, latency -25%, memory +15%.

Pattern 2 – TDD with Claude

Bối cảnh: spec rõ ràng (đầu vào, đầu ra, edge cases), độ phủ test cao là yêu cầu. Đây là tình huống lý tưởng cho Test-Driven Development.

Workflow – chu kỳ Red–Green–Refactor truyền thống, nhưng Claude đóng vai trò implementor:

- Viết test trước cho một hành vi cụ thể. Test fail vì chưa có implementation (Red).
- Yêu cầu Claude: “Triển khai function passing this test. Không sửa test, không sửa các function khác.”
- Claude sinh implementation. Chạy test ngay.
- Nếu pass (Green) → chuyển sang refactor (Refactor); nếu fail → paste output lỗi vào prompt để Claude sửa.
- Sau khi xanh + refactor xong, viết test kế tiếp cho hành vi tiếp theo. Lặp lại bước 2.

Đặc trưng: test là spec sống – không bị lệch như spec viết trên giấy. Tránh được tình trạng code nhìn đúng nhưng không pass behaviour vì test chính là behaviour đo lường được.

Tips thực hành:

- Viết test ngắn, độc lập – mỗi test cover đúng một hành vi.
- Yêu cầu Claude tuân thủ pattern Arrange-Act-Assert: ba khối rõ ràng trong mỗi test.
- Sau khi test pass, hỏi Claude “Có edge case nào còn thiếu không?” rồi viết test cho edge case đó. Đây là cách dùng Claude làm reviewer của test suite.

Ví dụ ngắn: xây hàm `parse_iso_date(s)` parse string ISO 8601 thành `datetime`. Test 1: parse “2026-01-15” → test pass sau khi Claude triển khai. Test 2: parse “2026-01-15T10:30:00Z” → Claude phát hiện phần triển khai thiếu xử lý timezone, fix. Test 3: parse chuỗi rỗng → Claude raise `ValueError`. Test 4: edge case “2026-02-29” (năm không nhuận) → Claude raise `ValueError`. Sau 4 test, hàm đạt coverage 100% và đã handle các edge case quan trọng.

Pattern 3 – Plan-Driven Development

Bối cảnh: spec mơ hồ, không có test sẵn, và task có rủi ro cao (vd thêm tính năng mới ảnh hưởng nhiều module, refactor cấu trúc database, thay đổi public API). Đây là tình huống mà code trước, xem sau rất nguy hiểm.

Workflow:

1. Vào Plan Mode (`(Shift+Tab) × 2`).
2. Mô tả mục tiêu cấp cao kèm ràng buộc: “không break compatibility”, “performance không giảm > 5%”, “phải có migration script”...
3. Yêu cầu Claude đề xuất 2–3 phương án khác nhau, mỗi phương án nêu trade-off riêng (vd phương án A nhanh nhưng tốn memory, phương án B tiết kiệm memory nhưng phức tạp hơn).
4. Thảo luận trade-off với Claude. Đặt câu hỏi: “Phương án nào dễ rollback nhất?”, “Phương án nào team hiện tại có kỹ năng để maintain?”. Chọn một phương án.
5. Yêu cầu Claude chi tiết hoá phương án đã chọn thành plan với danh sách file cần sửa, step list, và tiêu chí thành công.
6. Approve plan. Thoát Plan Mode.
7. Code theo plan từng step. Sau mỗi step, đối chiếu với tiêu chí thành công đã đặt.

Đặc trưng: thời gian dành cho plan có thể chiếm 30–50% tổng thời gian – nhưng đổi lại code sinh ra ít có khả năng sai hướng. Phù hợp khi cost của một bug rất cao (vd hệ thống thanh toán, dữ liệu y tế).

Điểm cần lưu ý: tránh plan paralysis. Nếu sau 3 vòng thảo luận plan vẫn không hội tụ, có nghĩa scope chưa được khoanh đủ. Lúc đó dừng lại, viết `SPEC.md` ngắn để đóng băng quyết định, hoặc chia task thành các phần nhỏ hơn rồi áp dụng Plan-driven cho từng phần.

Ví dụ ngắn: refactor authentication system từ session-cookie sang JWT. Phase plan: Claude đề xuất ba phương án (rolling rollout, big-bang, hybrid). Discuss: rolling rollout cần 4 tuần và infrastructure phức tạp; big-bang một weekend nhưng risky; hybrid (JWT cho user mới, session cho user cũ trong 6 tháng) là compromise hợp lý. Chọn hybrid. Claude detail plan: 7 file backend, 3 file frontend, 2 migration script. Code phase: tuân từng step, mỗi step có test riêng. Tổng thời gian: 1 tuần (plan 2 ngày + code 3 ngày).

Pattern 4 – Spec-First Development

Bối cảnh: tính năng mới rõ ràng, ít test sẵn có – chúng ta đang xây từ đầu một module và muốn phần triển khai cùng với test được sinh ra một cách nhất quán từ cùng một nguồn.

Workflow:

1. Viết `SPEC.md` trước. Cấu trúc đề xuất gồm sáu phần: Goal (mục tiêu cấp cao), Inputs (định dạng đầu vào), Outputs (định dạng đầu ra), Edge cases (các trường hợp biên), Performance constraints (ràng buộc hiệu năng), Acceptance criteria (tiêu chí nghiệm thu).
2. Yêu cầu Claude: “Đọc `@SPEC.md`. Triển khai module + sinh test cover mọi acceptance criteria.”
3. Claude sinh module + test file đồng thời.
4. Chạy test. Nếu pass → xong; nếu fail → paste output cho Claude debug.
5. Nếu phát hiện edge case mới chưa có trong SPEC, cập nhật `SPEC.md` trước rồi yêu cầu Claude cập nhật cả code và test theo SPEC mới. Nguyên tắc: SPEC là source of truth, code và test theo SPEC chứ không ngược lại.

Đặc trưng: `SPEC.md` trở thành source of truth – code và test được sinh từ cùng một nguồn nên đồng bộ. Hữu ích cho team onboarding: dev mới đọc `SPEC.md` là hiểu được module mà không cần đọc code.

Điểm cần lưu ý: `SPEC.md` phải đủ chi tiết về acceptance criteria. Nếu viết “function should be efficient” thì Claude không có gì để test against. Quy tắc thực dụng: mỗi criteria phải đo lường được – thay vì “efficient”, viết “p95 latency < 50ms cho input 1000 phần tử”.

Ví dụ ngắn: xây module rate limiter mới. `SPEC.md` có: Goal (limit request per user), Inputs (`user_id`, `action`), Outputs (allow/deny + remaining quota), Edge cases (concurrent request, clock skew, Redis down), Performance ($p95 < 10\text{ms}$, throughput > 10k QPS), Acceptance criteria (10 tiêu chí cụ thể). Claude sinh `rate_limiter.py` + `test_rate_limiter.py` đồng thời, mỗi acceptance criteria có một test tương ứng. Pass ngay lần đầu vì SPEC chi tiết.

Khi nào chuyển giữa các pattern

Bốn pattern trên không cố định cho cả dự án. Trong một dự án dài, ta thường chuyển qua lại theo các giai đoạn:

- **Giai đoạn khởi đầu** (phạm vi chưa rõ): áp dụng *Plan-driven* để khoanh vùng và quyết định approach.
- **Giai đoạn xây dựng module mới** (đã có spec sơ bộ): chuyển sang *Spec-first* để sinh phần triển khai đầu tiên cùng test một lúc.
- **Giai đoạn hoàn thiện** (đã có code, cần test bao phủ): chuyển sang *TDD with Claude* để thêm test cho các edge case còn lại.
- **Giai đoạn maintain dài hạn** (mã nguồn ổn định): áp dụng *Iterative* cho các điều chỉnh nhỏ, tối ưu performance, sửa bug.

Một dự án vibe coding chuyên nghiệp sẽ thấy cả bốn pattern được áp dụng ở các giai đoạn khác nhau – không pattern nào tốt nhất tuyệt đối, mỗi pattern là một công cụ cho một loại bài toán cụ thể. Khả năng nhận diện đúng giai đoạn và chuyển pattern kịp thời là kỹ năng phân biệt giữa người mới và developer thành thạo vibe coding.

VIII. Mở rộng Claude Code: Skill, Subagent, MCP, Hook, Custom Slash Command

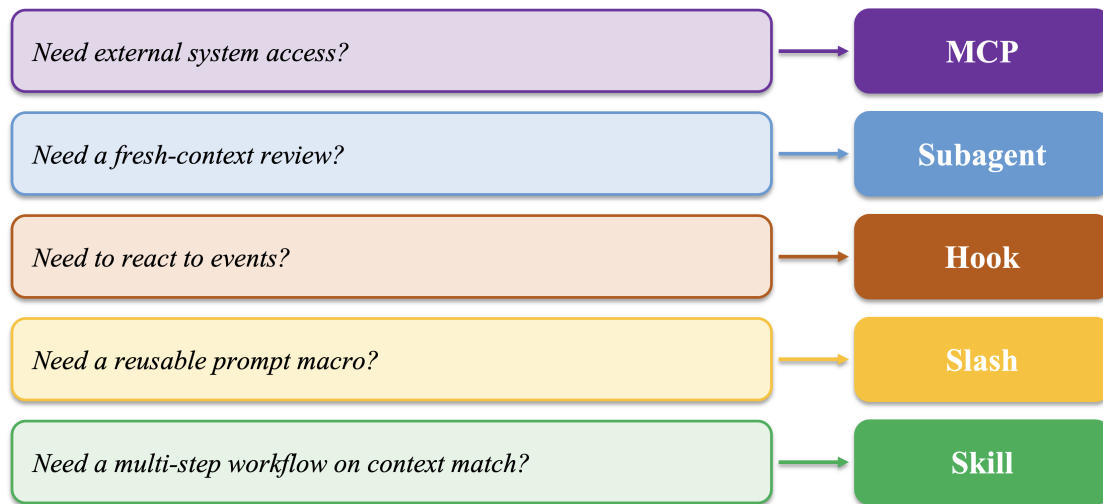
Đến đây chúng ta đã có một agent và biết cách quản lý context. Tuy nhiên, vibe coding trong dự án thật luôn đối mặt với những bài toán lặp đi lặp lại mà Claude Code mặc định không giải quyết tối ưu: code review tự động, tích hợp tool ngoài, format nội dung khi save, prompt template tái sử dụng. Anthropic giải quyết các bài toán này bằng năm cơ chế mở rộng – mỗi cơ chế là một điểm tích hợp khác nhau giữa Claude và môi trường lập trình. Phần này giới thiệu các cơ chế này, với tiêu chí lựa chọn rõ ràng.

VIII.1. Tổng quan các cơ chế mở rộng

Claude Code có năm cơ chế mở rộng chính thức: **Skill**, **Subagent**, **MCP**, **Hook**, và **Slash command**. Mỗi cơ chế tối ưu cho một loại task khác nhau – chúng không thay thế cho nhau mà bổ trợ. Bảng dưới so sánh năm cơ chế trên các tiêu chí: cách trigger, phạm vi tác động, và use case điển hình:

Cơ chế	Trigger	Phạm vi	Use case
Skill	Description khớp prompt	Workflow N-bước	Auto-load debug guide
Subagent	User invocation	Specialist task	Code review, audit
MCP	Tool call	External system	Database, API, custom server
Hook	Event (pre/post edit)	Side effect	Format on save, log diffs
Slash	Manual /cmd	Macro	Project-specific shortcut

Khi chúng ta không chắc nên chọn cơ chế nào, cây quyết định dưới đây giúp khoanh vùng nhanh chỉ bằng 1–2 câu hỏi:

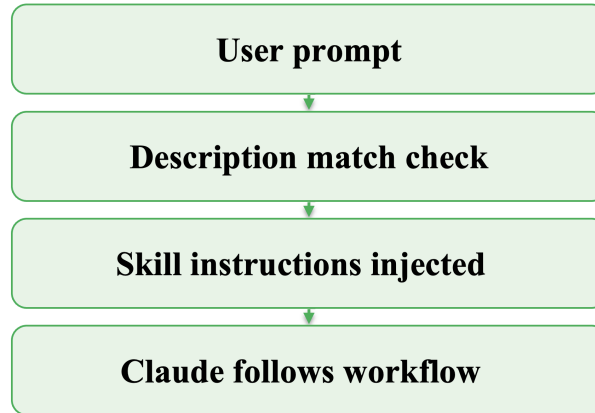


Hình 20: Cách lựa chọn các chiến lược mở rộng dựa vào xác định nhu cầu (external system, fresh review, event reaction, prompt macro, hay multi-step workflow).

VIII.2. Skills – Workflow Auto-trigger

Một Skill là một folder `.claude/skills/<name>/` chứa file `SKILL.md`. Đặc biệt nhất: Skill *tự load* khi description khớp với prompt – ta không cần gõ tên skill.

- **Vị trí:** `.claude/skills/<name>/SKILL.md` – nội dung là một file Markdown với YAML frontmatter (name, description) ở đầu và workflow N bước ở thân.
- **Hai trigger style:**
 - *Auto-trigger*: description khớp prompt → Claude Code load âm thầm, chèn workflow vào prompt. Đây là chế độ chủ đạo.
 - *Manual*: `/skills <name>` – ít dùng, chủ yếu để test skill.
- **Auto-trigger flow:** (1) user prompt → (2) Claude matching description → (3) skill instructions được chèn vào prompt → (4) Claude theo workflow.
- **Ví dụ:** bạn nói “my agent loop is stuck on iteration 5”, Skill `react-loop-debug` tự load và inject 4 bước debug pattern đã được team encode sẵn.
- **Phù hợp nhất:** workflow lặp lại có N bước cố định – review code, debug pattern, deploy checklist.

Auto-trigger flow*Insight:*

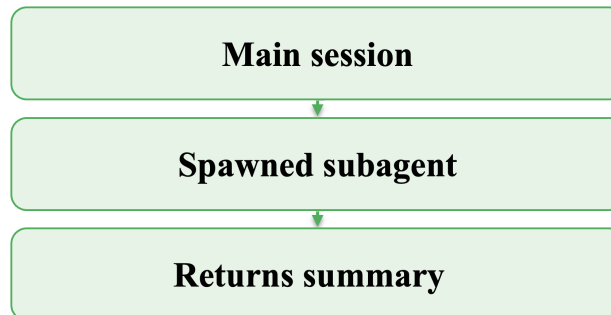
Skills convert your tribal knowledge into Claude's muscle memory.

Hình 21: Sơ đồ Skills – auto-trigger flow từ user prompt đến workflow được inject.

Skill được thiết kế để lưu trữ tri thức của team thành workflow tự động. Khi developer mô tả tự nhiên một vấn đề (vd “agent loop bị infinite”), Claude Code matching description của các skill có sẵn, load skill phù hợp vào prompt, và follow workflow đã được encode. Khác với Slash command (cần nhớ tên), Skill tự kích hoạt – đây là điểm khiến nó trở thành kỹ năng quan trọng.

VIII.3. Subagents – Chuyên biệt với Context Riêng

Skill phù hợp khi cần một workflow chuẩn cho cùng main agent thực hiện. Nhưng đôi khi vấn đề lại nằm ở chỗ ngược lại: ta cần *một agent khác, với góc nhìn khác*, để giải task một cách độc lập. Đây là lúc Subagent xuất hiện.

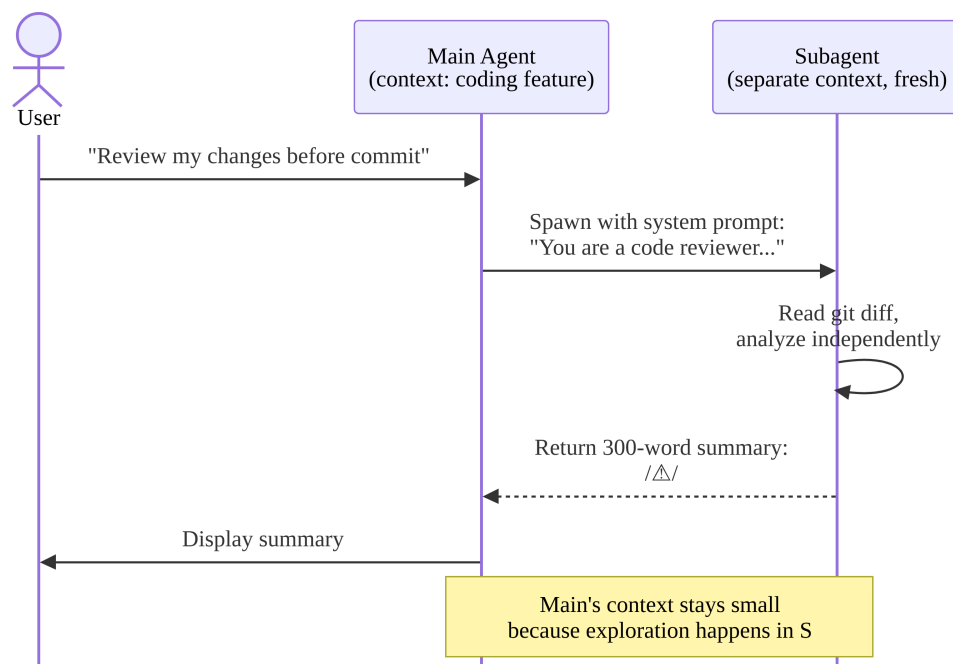
Spawn pattern*Insight:*

Use a subagent when fresh eyes matter more than session memory.

Hình 22: Subagent architecture. Subagent có context riêng – đọc, suy nghĩ nhưng chỉ trả summary ngắn về main.

Subagent là một agent con được sinh ra cho một task cụ thể, có *context window riêng* – không làm phình context của main session, và trả về chỉ một summary ngắn gọn. Trong thực tế, đây là cơ chế gần nhất với việc nhờ đồng nghiệp review code: ta đưa cho họ phạm vi nhỏ, họ làm việc trong môi trường riêng, kết quả trả về dưới dạng summary.

- **Vị trí:** `.claude/agents/<name>.md`
- **Context window riêng** → không ảnh hưởng main session.
- **Tool list bị giới hạn:** vd reviewer = [Read, Grep, Bash] (không có Edit) → chỉ đọc, không sửa.
- **Use case kinh điển:** code review trước commit. “Use agent-reviewer to check changes since last commit.”



Hình 23: Sơ đồ chi tiết Subagent – main agent đẩy task xuống subagent với context riêng, subagent xử lý độc lập rồi trả về summary ngắn.

Sơ đồ trên minh họa cùng một concept ở hai mức độ chi tiết. Sơ đồ tổng quan thể hiện quan hệ: main agent khởi tạo subagent, subagent có context riêng, hoàn thành task, trả về summary ngắn. Sơ đồ chi tiết hơn cho thấy luồng cụ thể – main đẩy task xuống subagent, subagent đọc/grep/analyze trong context riêng (không làm phình context của main), sau đó tổng hợp kết quả thành một summary gọn. Pattern này đặc biệt phù hợp cho code review trước commit, security audit, hoặc bất kỳ task nào cần góc nhìn mới mẻ không bị bias bởi quá trình code lâu trước đó.

Format file subagent `.claude/agents/agent-reviewer.md`

```

1 ---
2 name: agent-reviewer
3 description: Review code của ReAct agent.
4   Use khi user nhắc "review" hoặc trước commit.
5 tools: [Read, Grep, Bash]
6 ---
7
8 You are a senior Python code reviewer specialized in
9 ReAct agent implementations.
10
11 ## Review checklist
12 1. Calculator không dùng eval/exec (whitelist ast)
13 2. ReAct loop có MAX_ITERATIONS
14 3. Type hints + docstring đầy đủ
15 4. Append assistant message + tool_result đúng thứ tự
16 5. Tool descriptions rõ về format input
17 6. JSON parse/serialize cho tool_result
18
19 ## Output format
20 + <file:line> - what's good
21 ~ <file:line> - suggestion
22 - <file:line> - must fix
23
24 ## Important
25 - Read-only. Do NOT edit files.
26 - Report concisely, <= 400 words.

```

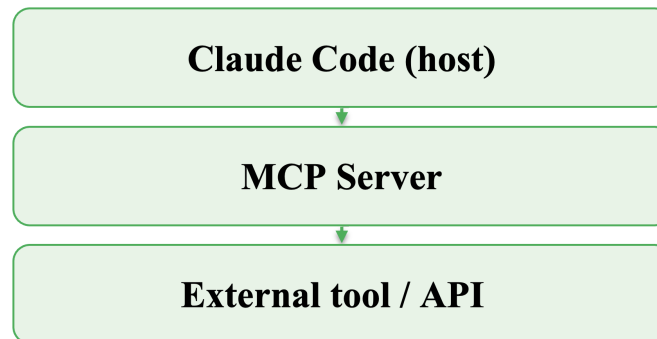
VIII.4. MCP – Model Context Protocol

Skill và Subagent đều ở trong Claude Code: chúng là folder Markdown nằm trong `.claude/`. Cơ chế thứ ba mà chúng ta xem xét là cách mở rộng Claude Code với các công cụ bên ngoài. MCP (Model Context Protocol) là chuẩn JSON-RPC do Anthropic định nghĩa, cho phép external program (database, search API, internal microservice, custom script) cung cấp tool cho Claude Code. Giao thức có thể chạy qua stdio (tiêu chuẩn, dùng cho local server) hoặc HTTP (dùng cho remote service). Khả năng này biến Claude Code từ một REPL đóng thành một host plugin – mọi tool trong hệ sinh thái phần mềm đều có thể trở thành một tool của agent.

- **Cấu hình:** thêm vào `~/.claude.json`, mục `mcpServers`.
- **Cho phép:** external program (vd Tavily, GitHub, database, custom script) cung cấp tool theo chuẩn chung.
- **Hai mối quan hệ:**
 - *Sử dụng:* cài MCP server của người khác (vd Tavily MCP).
 - *Tự xây:* tự đóng gói script của bạn thành MCP server (Lab 2 Phase 4).

Lưu ý quan trọng – MCP vs REST API

MCP là cơ chế cho **Claude Code (host)** – khi chúng ta cài MCP server vào `~/.claude.json`, đó là gắn tool cho Claude Code REPL, không phải cho Python agent của bạn. Python agent code (như ReAct Agent ở Lab 1, Lab 2) gọi REST API trực tiếp qua `httpx` – vì nó không phải Claude Code. Phần này sẽ được làm rõ thêm ở Lab 2.

MCP architecture***Insight:***

MCP is for Claude Code — your Python agent code uses REST API directly.

Hình 24: MCP architecture. Claude Code (client) tương tác với nhiều MCP server theo cùng một protocol.

Sơ đồ MCP nhấn mạnh khả năng nhân rộng tool của Claude Code thông qua external server. Mỗi MCP server (Tavily, GitHub, internal database...) expose tool theo cùng một giao thức JSON-RPC. Từ phía Claude Code, các tool từ MCP server hiển thị giống như built-in tool – LLM gọi chúng bằng tool call thông thường. Sự khác biệt chỉ nằm ở chỗ thực thi: thay vì host xử lý trực tiếp, host forward request qua stdio hoặc HTTP đến MCP server bên ngoài, đợi response, rồi đưa kết quả về cho LLM. Cơ chế này biến Claude Code thành một host plugin mở – mọi tool trong hệ sinh thái phần mềm đều có thể trở thành tool của agent miễn là có MCP wrapper.

VIII.5. Hooks – Event-driven Automation

Skill, Subagent, MCP đều hoạt động theo mô hình chủ động: agent quyết định khi nào load skill, spawn subagent, gọi MCP tool. Hooks là cơ chế thứ tư đảo chiều quan hệ – thay vì agent gọi, sự kiện trong môi trường gọi agent. Hooks là cơ chế *event-driven*: đăng ký một command shell để chạy khi có event cụ thể xảy ra (trước/sau edit, khi start session, khi save file). Đây là nơi để *tự động hoá các tác động phụ* – những việc lặp đi lặp lại mà bạn muốn diễn ra ngầm không phải nghĩ đến: format code sau mỗi edit, log diff vào audit file, chạy test trước commit.

- **Events thường dùng:** PreEdit, PostEdit, SessionStart.

- **Use case phổ biến:**
 - Tự định dạng Python với `black` sau mỗi edit.
 - Log mọi diff vào file audit.
 - Chạy `pytest` trước commit.
- **Cấu hình:** trong `~/.claude.json`, mục `hooks`.
- **Cẩn thận:** hooks chặn edit (vd fail format) → mất flow, bực mình.

Ví dụ cấu hình hook trong `/.claude.json`

```

1 {
2   "hooks": {
3     "PostEdit": [
4       {
5         "command": "black ${file}",
6         "filter": "*.py"
7       }
8     ],
9     "SessionStart": [
10      {"command": "git pull --rebase"}
11    ]
12  }
13 }
```

VIII.6. Custom Slash Commands

Cơ chế cuối cùng là cơ chế đơn giản nhất – và vì thế cũng hữu dụng nhất cho việc bắt đầu. Slash command về bản chất là một prompt template được lưu lại và đặt tên. Slash command là bookmark của prompt – giúp bạn không phải gõ lại cùng một đoạn dài mỗi lần thực hiện workflow lặp. Cụ thể, Slash command là một file Markdown trong `.claude/commands/<name>.md` chứa nội dung prompt, gọi bằng `/<name>` trong REPL.

- **Vị trí:** `.claude/commands/<name>.md`
- **Nội dung:** prompt template tái sử dụng.
- **Invoke:** gõ `/<name>` trong REPL.
- **Use case:**
 - `/review-pr` – checklist review của team.
 - `/seed-tests` – sinh test cho file selected.
 - `/onboard` – script onboard người mới.

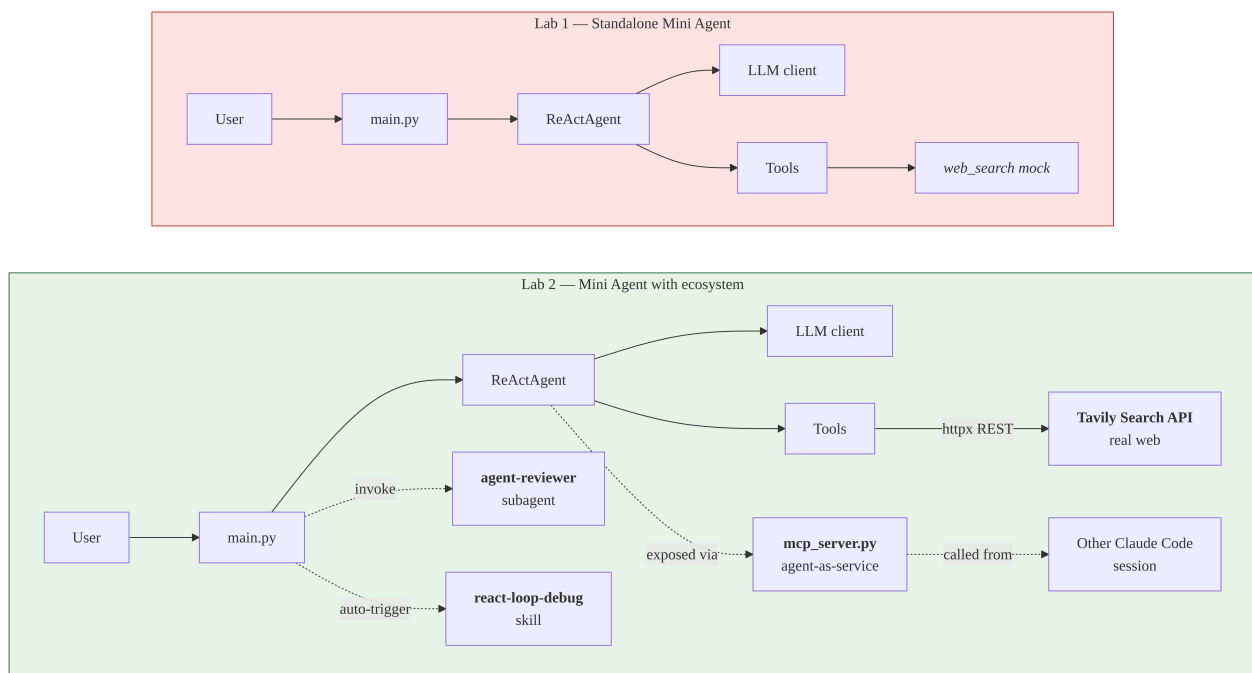
.claude/commands/review-pr.md

```
1 Review the staged changes against this checklist:
2
3 1. Type hints on every public function
4 2. Tests for new behavior
5 3. CLAUDE.md conventions followed
6 4. No console.log / print() left in
7
8 Report + / ~ / - per item.
```

IX. Lab 2 – Mở rộng ReAct Agent với MCP, Subagent, Skill

IX.1. Mục tiêu Lab 2

Lab 1 đã cho chúng ta một agent chạy được nhưng nhìn kỹ thì agent đó có ba hạn chế lớn: (a) `web_search` là mock, không trả lời được câu hỏi thật về thông tin hiện tại; (b) không có review pipeline – code commit thẳng không có ý kiến phản biện; (c) không có hỗ trợ debug khi gặp lỗi agent loop. Lab 2 giải quyết cả ba vấn đề trên qua ba phase tương ứng, mỗi phase áp dụng một cơ chế từ – MCP, Subagent, Skill. Phase 4: biến chính agent thành một MCP server cho *Claude Code* khác gọi vào.



Hình 25: So sánh kiến trúc của 2 thiết kế trong Lab 1 và Lab 2.

Sau Lab 2, chúng ta có:

- **Web search** – thay `web_search` mock bằng Tavily Search REST API. Agent giờ truy cập internet được.
- **Subagent custom** – build `agent-reviewer` chuyên review code agent theo convention CLAUDE.md.
- **Skill custom** – tạo `react-loop-debug` auto-trigger khi user gặp lỗi agent loop.
- **MCP server** – cung cấp ReAct Agent dưới dạng tool, biến agent thành “service” mà bất kỳ Claude Code nào cũng gọi được.

IX.2. Phase 1 – Web search với Tavily REST API

Phase này thay `web_search` mock bằng search qua Tavily Search API. Sau phase, agent trả lời được câu hỏi cần thông tin internet như “Giá Bitcoin hôm nay?”, “Tổng thống Mỹ hiện tại đắc cử năm nào?”.

Bước 1 – Lấy Tavily Search API key

Đầu tiên cần một Tavily API key:

- Vào tavily.com, đăng ký tài khoản (Google login OK).
- Vào Dashboard → free tier mặc định 1,000 query/tháng (đủ cho Lab).
- Copy API key (dạng `tvly-...`) vào file text an toàn trước – sẽ paste vào `.env` ở bước kế.

Bước 2 – Lưu API key vào `.env`

Trong folder `mini-agent`, mở `.env` (file đã tạo từ Lab 1 chứa `OPENAI_API_KEY`). Thêm dòng:

```
.env
1 OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
2 TAVILY_API_KEY=tvly-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Bước 3 – Refactor `tools.py` qua workflow $E \rightarrow P \rightarrow C \rightarrow C$

API key sẵn sàng. Giờ refactor ReAct Agent thay `web_search` mock bằng gọi Tavily thật, đi qua workflow chuẩn:

(1) Vào Claude Code: `cd ~/aio-vibe-coding/mini-agent && claude`. (2) Vào Plan Mode bằng `Shift+Tab` ×2. (3) Yêu cầu plan:

```
Prompt yêu cầu plan
1 Đọc @CLAUDE.md và @tools.py. Đề xuất plan để thay
2 hàm web_search mock bằng gọi Tavily Search API thật.
3
4 Yêu cầu:
5 - Dùng httpx (sync) POST tới https://api.tavily.com/search
6 - Đọc TAVILY_API_KEY từ os.getenv, đã trong .env
7 - Giữ signature web_search(query: str) -> dict
8 - Output vẫn theo format cũ: {"result": str, "error": str | None}
9   trong đó result là chuỗi tổng hợp 3-5 kết quả top
10 - Handle network error (timeout, rate limit, missing key)
11 - Update test_tools.py: mock httpx response cho test offline
12
13 Plan only, không code.
```

(4) Claude đề xuất plan sau ~30 giây. Review checklist: có giữ signature `web_search(query)` -> dict? (để không sửa `agent.py`); có handle `TAVILY_API_KEY` missing rõ ràng?; có timeout explicit (vd 15s)?; mock test có hợp lý, không cần network khi chạy `pytest`?

(5) Approve plan, thoát Plan Mode, để Claude code. Trong Approval Mode, review từng diff – đặc biệt error handling và parse Tavily response. Hàm `web_search` sau refactor trông như sau:

tools.py – web_search qua Tavily REST API

```

1 import os
2 import httpx
3
4 TAVILY_ENDPOINT = "https://api.tavily.com/search"
5
6
7 def web_search(query: str) -> dict:
8     """Search the web via Tavily API."""
9     api_key = os.getenv("TAVILY_API_KEY")
10    if not api_key:
11        return {"result": "", "error": "TAVILY_API_KEY missing in .env"}
12
13    try:
14        resp = httpx.post(
15            TAVILY_ENDPOINT,
16            json={
17                "api_key": api_key,
18                "query": query,
19                "max_results": 5,
20                "search_depth": "basic",
21            },
22            timeout=15.0,
23        )
24        resp.raise_for_status()
25        data = resp.json()
26    except httpx.TimeoutException:
27        return {"result": "", "error": "Tavily timeout (15s)"}
28    except httpx.HTTPStatusError as e:
29        return {"result": "", "error": f"Tavily HTTP {e.response.status_code}"}
30    except Exception as e:
31        return {"result": "", "error": f"Tavily error: {e}"}
32
33    results = data.get("results", [])
34    if not results:
35        return {"result": "No results found.", "error": None}
36
37    formatted = "\n\n".join(
38        f"- {r['title']}\n {r.get('content', '')[:200]}"
39        for r in results[:5]
40    )
41    return {"result": formatted, "error": None}

```

Bước 4 – Verify Phase 1

Test agent với câu hỏi cần search thật:

Output expected

```

1 $ python main.py "Giá Bitcoin hôm nay khoảng bao nhiêu USD?"
2
3 [Iter 1] Action: web_search({'query': 'current Bitcoin price USD'})
4 [Iter 1] Observation: - Bitcoin Price | BTC USD Live Chart - CoinDesk
5   Bitcoin is trading at $103,420 USD as of...
6   - Bitcoin (BTC) Live Price Today - CoinGecko
7   Bitcoin price today is $103,418 with a 24-hour trading volume...
8
9 [Iter 2] Action: final_answer({'answer': 'Giá Bitcoin hiện khoảng 103,400 USD.'})
10
11 [Final] Answer: Giá Bitcoin hiện khoảng 103,400 USD.
```

Phase 1 done. Commit:

```

1 $ git add .
2 $ git commit -m "feat(tools): replace web_search mock with Tavily REST API"
```

IX.3. Phase 2 – Subagent agent-reviewer

Sau Phase 1, agent đã có thể trả lời câu hỏi cần thông tin internet. Tuy nhiên, mỗi lần refactor như vậy đặt ra một bài toán mới: *ai review code?*. Trong workflow truyền thống, một pull request được senior dev review trước khi merge. Trong vibe coding, vai trò này có thể được delegate cho một subagent có context riêng và checklist riêng. Phase 2 tạo subagent custom **agent-reviewer** – chuyên review code Agent, áp dụng convention từ **CLAUDE.md** và các anti-pattern đã học ở Lab 1. Đây là lần đầu tiên trong khoá học chúng ta tách một phần ra khỏi main agent.

Bước 1 – Tạo file subagent

Trong REPL Claude Code (Plan Mode), yêu cầu Claude tạo file:

Prompt tạo subagent

```

1 Tạo subagent custom tại .claude/agents/agent-reviewer.md.
2
3 Yêu cầu:
4 - name: agent-reviewer
5 - description: Review code của ReAct agent. Use khi
6   user nhắc "review", "review agent code", trước commit
7 - tools: [Read, Grep, Bash] read-only
8 - System prompt: senior Python reviewer cho agent code
9
10 Checklist review (theo CLAUDE.md và anti-patterns Lab 1):
11 1. Calculator KHÔNG dùng eval/exec
12 2. ReAct loop có max_iterations
13 3. Type hints + docstring đầy đủ
14 4. Append assistant message + tool_result đúng thứ tự
15 5. Tool descriptions rõ, không mơ hồ
16 6. JSON parse/serialize cho tool_result (không str(dict))
17
18 Output format: + / ~ / - với cụ thể line/file.

```

Claude đề xuất plan ngắn (chỉ tạo 1 file), approve. Lưu ý quan trọng: tools chỉ [Read, Grep, Bash] – không có Edit, để đảm bảo subagent không thể tự sửa code trong quá trình review.

Bước 2 – Test subagent

Gọi subagent review những thay đổi của Phase 1:

Trong Claude Code REPL

```

1 > Use the agent-reviewer subagent to review changes since
2   the previous commit (Phase 1 - Tavily Search integration).

```

Subagent sẽ được khởi tạo (context riêng), đọc git diff, chạy checklist, và trả về report:

Output mong đợi

```

1 Subagent agent-reviewer report:
2
3 + tools.py:45 - signature web_search(query: str) -> dict preserved
4 + tools.py:48 - try/except cover httpx errors
5 + tools.py:52 - type hints đầy đủ
6 ~ tools.py:57 - Tavily timeout 15s, có thể parameterize qua env
7 ~ test_tools.py:23 - mock chưa test case TAVILY_API_KEY missing
8 - tools.py:62 - empty results chưa wrap theo format
9   {"result": ..., "error": ...} khi data["results"] rỗng

```

Phát hiện 1 lỗi (-) phải fix. Sửa, chạy lại reviewer, lần này sạch. Commit Phase 2:

```

1 $ git add .
2 $ git commit -m "feat: add agent-reviewer subagent with project-specific checklist"

```

Vì sao Subagent (không phải Skill) cho task review code

Có ba lý do:

- (1) **Context isolation** – main agent đã đọc code lâu, có bias “code này tôi viết, chắc chắn đúng”; subagent có context riêng nên có góc nhìn mới mẻ.
- (2) **Tool restriction** – subagent chỉ có [Read, Grep, Bash], không thể tự sửa, giúp bạn yên tâm khi delegate.
- (3) **Concise output** – subagent trả về summary ≤ 400 words, không pollute main context với chi tiết review trung gian.

IX.4. Phase 3 – Skill react-loop-debug

Phase 1 và Phase 2 đã giải quyết hai trong ba hạn chế của Lab 1: search và review pipeline. Còn lại là hạn chế thứ ba – thiếu hỗ trợ debug khi gặp lỗi agent loop. Phase này tạo Skill **react-loop-debug** đóng gói bốn pattern debug đã học ở Lab 1 thành một workflow có thể tự kích hoạt. Khác với hai cơ chế trước, Skill nổi bật ở khả năng *auto-trigger*: user không cần nhớ tên skill, chỉ cần nói tự nhiên về bug, Claude Code tự nhận diện description match và load skill vào prompt.

Bước 1 – Tạo Skill folder và SKILL.md

Prompt Claude Code tạo skill:

Prompt tạo skill

```

1 Tạo skill tại .claude/skills/react-loop-debug/SKILL.md.
2
3 Description trigger: khi user nhắc "agent loop bug", "infinite
4 loop", "agent stuck", "iteration limit", hoặc các từ khóa
5 liên quan đến debug ReAct agent.
6
7 Workflow (4 bước check theo Chương 4 textbook):
8 1. Check messages array: assistant message có được append?
9 2. Check tool_use_id matching giữa tool_use và tool_result
10 3. Check tool descriptions: LLM gửi input format có khớp không
11 4. Check MAX_ITERATIONS có set không (mặc định 10)
12
13 Mỗi bước hướng dẫn:
14 - Lệnh để verify
15 - Symptom của lỗi
16 - Cách fix
17
18 allowed-tools: [Read, Edit, Bash]

```

Claude tạo skill folder + SKILL.md với nội dung như sau:

.claude/skills/react-loop-debug/SKILL.md

```

1 ---
2 name: react-loop-debug
3 description: Debug ReAct agent loop khi gặp infinite loop, agent
4   stuck, iteration limit exceeded, hoặc tool call errors.
5 allowed-tools: [Read, Edit, Bash]
6 ---
7
8 # ReAct Loop Debug Workflow
9
10 ## Pattern 1: Infinite loop - cùng tool mỗi iteration
11 Symptom: agent chạy đến MAX_ITERATIONS rồi raise.
12 Check: grep -n "messages.append" agent.py
13 Fix: phải append response.content (assistant message)
14   sau mỗi LLM response.
15
16 ## Pattern 2: tool_use_id mismatch (400 Bad Request)
17 Symptom: lần gọi LLM thứ 2 báo 400.
18 Check: print tool_call_id từ response và từ tool_result.
19 Fix: dùng call.id từ LLM response, không tự đặt UUID.
20
21 ## Pattern 3: Tool description mơ hồ
22 Symptom: tool báo SyntaxError, ValueError.
23 Check: docstring có rõ format input không?
24 Fix: refactor docstring rõ format (vd "pure arithmetic, no variables").
25
26 ## Pattern 4: Không có MAX_ITERATIONS
27 Symptom: agent chạy mãi.
28 Check: grep -n "MAX_ITERATIONS|max_iter" agent.py
29 Fix: thêm MAX_ITERATIONS = 10 và check trong loop.

```

Bước 2 – Test skill auto-trigger

Sau khi tạo, reset session (/clear) để Claude load skill mới. Test bằng cách mô tả lỗi tự nhiên – chú ý: không gõ /skill react-loop-debug, chỉ nói chuyện bình thường:

Test auto-trigger

```

1 > Agent loop của tôi infinite - LLM gọi cùng calculator
2 ("2026 * 3") mỗi iteration đến hit MAX_ITERATIONS. Help?
3
4 [Loaded skill: react-loop-debug]
5
6 Theo workflow debug, symptom của bạn khớp Pattern 1 -
7 infinite loop với cùng tool. Likely cause: thiếu
8 append assistant message.
9
10 Kiểm tra:
11 grep -n "messages.append" agent.py
12
13 -> Found: chỉ append tool_result, KHÔNG append response.content
14 (assistant message).
15
16 Fix: thêm dòng sau khi nhận response từ LLM:
17 messages.append({"role": "assistant", "content": response.content})
18
19 Có muốn tôi apply fix không?

```

Đây là ví dụ thú vị đầu tiên với Skill: bạn không gõ `/skill react-loop-debug` – bạn chỉ mô tả tự nhiên về bug, Skill tự đến và inject workflow debug vào prompt. Skill auto-trigger thành công. Commit Phase 3:

```

1 $ git add .
2 $ git commit -m "feat: add react-loop-debug skill with 4-pattern checklist"

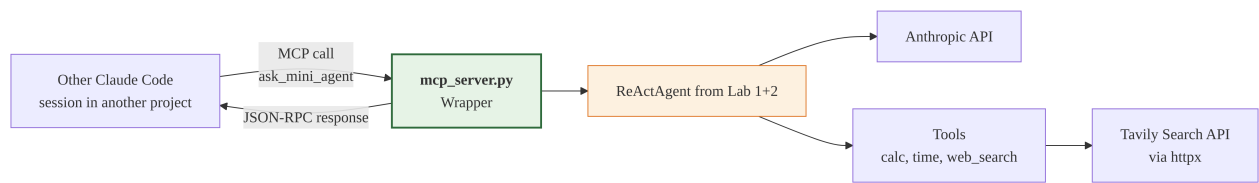
```

IX.5. Phase 4 – Build MCP Server (Agent-as-a-Service)

Sau ba phase đầu, ReAct Agent đã đầy đủ tính năng cho mục tiêu cá nhân – search, review pipeline, debug auto-trigger. Nhưng nếu nhìn sâu hơn, một câu hỏi mới xuất hiện: *làm sao chia sẻ tính năng này cho phần còn lại của codebase?* Mỗi project khác bạn làm trên máy có thể cần khả năng multi-step reasoning với search + calculator – nhưng copy-paste folder `mini-agent` vào mỗi project rõ ràng không hợp lý. Phase 4: biến ReAct Agent thành một MCP server. Sau đó, bất kỳ Claude Code session nào trên máy bạn (hoặc team mate cài MCP) đều có thể gọi `ask_agent(question)` như một tool – agent của bạn trở thành *service*, không chỉ script.

Concept – Agent as a Service

Tại sao expose ReAct Agent thành MCP? Vì khi đó: (a) Claude Code đang code project khác có thể gọi `ask_mini_agent` để delegate task “cần multi-step reasoning với search + calculator”; (b) team mate cài MCP server của bạn vào `~/.claude.json`, dùng được agent với mọi feature đã build; (c) mỗi agent custom thành một tool node trong ecosystem lớn hơn – pattern này gọi là *agent-as-a-service*.



Hình 26: Pattern agent-as-a-service: Other Claude Code session → mcp_server.py → ReActAgent.

Bước 1 – Cài MCP Python SDK

```

1 $ cd ~/aio-vibe-coding/mini-agent
2 $ source .venv/bin/activate
3 $ pip install mcp
4 $ pip freeze | grep -E '^(mcp|anthropic|python-dotenv)' \
5   > requirements.txt
  
```

Bước 2 – Build mcp_server.py

Yêu cầu Claude Code tạo file:

Prompt tạo MCP server

```

1 Tạo file mcp_server.py ở root project.
2
3 Yêu cầu:
4 - Dùng mcp.server.fastmcp.FastMCP (lib mcp đã cài)
5 - Server name: "mini-research-agent"
6 - Expose 1 tool: ask_agent(question: str) -> str
7 - ask_agent: init OpenAIClient, init ReActAgent, gọi run(question)
8 - Docstring rõ ràng cho ask_agent để LLM client biết cách dùng:
9   "Run a ReAct agent that can answer multi-step questions using
10    calculator, current time, and web search tools. Returns final
11    answer as a string."
12 - Error handling: wrap trong try/except, trả error message thân thiện
13 - Run với stdio transport (mặc định của FastMCP)
14
15 Plan trước, code sau.
  
```

Claude tạo file với cấu trúc kỳ vọng:

mcp_server.py – wrapper FastMCP cho ReAct Agent

```

1  """MCP server exposing ReAct Agent as a tool."""
2  from mcp.server.fastmcp import FastMCP
3  from agent import ReActAgent
4  from llm import OpenAIClient
5
6  mcp = FastMCP("mini-research-agent")
7
8  # Init một instance để reuse qua nhiều call
9  _client = OpenAIClient()
10 _agent = ReActAgent(_client)
11
12
13 @mcp.tool()
14 def ask_agent(question: str) -> str:
15     """Run a ReAct agent that can answer multi-step questions
16     using calculator, current year, and web search (via Tavily API).
17     Returns the final answer as a string.
18
19     Args:
20         question: Câu hỏi tự nhiên, có thể multi-step.
21     Returns:
22         Final answer dạng string.
23     """
24     try:
25         return _agent.run(question)
26     except Exception as e:
27         return f"Agent failed: {type(e).__name__}: {e}"
28
29
30 if __name__ == "__main__":
31     mcp.run() # stdio transport mặc định

```

Bước 3 – Test server standalone

```

1  $ python mcp_server.py
2  # Server chạy ở stdio mode đợi MCP client connect.
3  # Không có output -- nếu không crash là server OK. Ctrl+C để dừng.

```

Bước 4 – Cài vào Claude Code

Edit ~/.claude.json, thêm vào mcpServers:

```

1 {
2   "mcpServers": {
3     "tavily-search": { ... },
4     "mini-research-agent": {
5       "command": "python",
6       "args": [
7         "/Users/<your-name>/aio-vibe-coding/mini-agent/mcp_server.py"
8       ],
9       "env": {
10        "PYTHONPATH": "/Users/<your-name>/aio-vibe-coding/mini-agent",
11        "OPENAI_API_KEY": "<your-openai-key>",
12        "TAVILY_API_KEY": "<your-tavily-key>"
13      }
14    }
15  }
16 }

```

Dùng *full absolute path* (không *relative*). Cần truyền `OPENAI_API_KEY` và `TAVILY_API_KEY` qua `env` section vì MCP server không tự đọc được `.env` của user khi Claude Code khởi tạo nó.

Bước 5 – Test từ Claude Code session khác

Sau khi restart Claude Code, mở session ở *folder khác* (không phải `mini-agent`) để test cross-project usability:

```

1 > /mcp
2 Connected MCP servers:
3 - tavily-search (1 tool: ...)
4 - mini-research-agent (1 tool: ask_agent)
5
6 > Use the ask_agent tool from mini-research-agent MCP to answer:
7   "Năm hiện tại nhân ba bằng bao nhiêu?"
8
9 [MCP tool call] mini-research-agent.ask_agent("Năm hiện tại nhân ba bằng bao nhiêu?")
10 [Server side: ReAct Agent đang chạy ReAct loop...]
11 [MCP tool response]: "6078"
12
13 + Câu trả lời: 6078

```

Phase 4 hoàn thành. Bạn vừa biến agent của mình thành một service – Claude Code khác (ở folder nào, project nào trên máy) đều gọi được. Commit final:

```

1 $ git add .
2 $ git commit -m "feat: expose ReAct Agent as MCP server (mcp_server.py)"

```

💡 Insight – MCP chuyển đổi mọi script Python thành tool

Một khi chúng ta biết build MCP server, mọi script Python có giá trị đều có thể trở thành MCP tool cho Claude Code dùng. Đây là pattern mở rộng vô hạn – agent của bạn = một mảnh ghép trong ecosystem lớn hơn. Sau Phase 4, có thể nhìn lại các script utility bạn đang dùng riêng và tự hỏi: “Cái nào nên cung cấp qua MCP để team cùng dùng?”

IX.6. Integration Test End-to-End

Sau khi cả bốn phase đã xong, chạy một câu hỏi multi-step kết hợp search + math + final:

Integration test

```

1 $ python main.py "Tổng thống Mỹ hiện tại đắc cử năm nào? Năm đó nhân 2 bằng bao nhiêu?"
2
3 [Iter 1] Action: web_search({'query': 'current US president elected year'})
4 [Iter 1] Observation: - Donald Trump elected 47th president in 2024...
5   - 2024 United States presidential election - Wikipedia
6     Trump won the 2024 election against Kamala Harris...
7
8 [Iter 2] Action: calculator({'expression': '2024 * 2'})
9 [Iter 2] Observation: 4048
10
11 [Iter 3] Action: final_answer({'answer': 'Đắc cử 2024, x 2 = 4048'})
12
13 [Final] Answer: Đắc cử 2024, x 2 = 4048
14 Final Result: Đắc cử 2024, x 2 = 4048

```

Đến đây chúng ta đã đi qua đủ các phần: định nghĩa vibe coding, cài đặt và workflow $E \rightarrow P \rightarrow C \rightarrow C$ (Mục III–V), tự build một ReAct Agent ở Lab 1, quản lý context (Mục VII), nắm cơ chế mở rộng (Mục VIII), và Lab 2 nâng cấp agent với MCP / Subagent / Skill.

Một số điểm cần lưu ý tổng kết như sau:

- Vibe coding không phải là AI viết hộ code. Developer vẫn đặc tả ý định, đọc diff, và chịu trách nhiệm cho mọi commit.
- Workflow $E \rightarrow P \rightarrow C \rightarrow C$ áp dụng được cho mọi task, từ một bug fix đơn giản đến refactor đa file. Skip phase nào cũng để lại lỗ hổng.
- CLAUDE.md là file đầu tư nhất — viết chi tiết file này sẽ giúp chúng ta tiết kiệm hàng giờ sửa diff lệch convention.
- Chọn cơ chế mở rộng phù hợp với task: Skill cho workflow N bước, Subagent khi cần context riêng, MCP để gắn tool ngoài, Hook cho side effect, Slash cho prompt template lặp.

X. Câu hỏi trắc nghiệm

1. (Khái niệm – Định nghĩa Vibe Coding)

Vibe Coding theo Karpathy được định nghĩa như thế nào?

- A. Một framework chính thức của Anthropic để dev viết code.
- B. Pattern hợp tác hai chiều người-AI, developer dẫn dắt ý định và AI sinh diff.
- C. Một tool tự động sinh toàn bộ mã nguồn từ spec markdown.
- D. Một IDE chuyên dụng cho lập trình bằng giọng nói.

2. (Khái niệm – Bốn phase workflow)

Thứ tự đúng của workflow $E \rightarrow P \rightarrow C \rightarrow C$ là gì?

- A. Code $\rightarrow$ Plan $\rightarrow$ Explore $\rightarrow$ Commit.
- B. Explore $\rightarrow$ Plan $\rightarrow$ Code $\rightarrow$ Commit.
- C. Plan $\rightarrow$ Explore $\rightarrow$ Commit $\rightarrow$ Code.
- D. Explore $\rightarrow$ Code $\rightarrow$ Plan $\rightarrow$ Commit.

3. (Implementation – Phím tắt Plan Mode)

Để vào Plan Mode trong Claude Code, ta bấm tổ hợp phím nào?

- A. `Ctrl` + `P`.
- B. `Shift` + `Tab` một lần.
- C. `Shift` + `Tab` hai lần.
- D. `Esc` + `Esc`.

4. (Khái niệm – CLAUDE.md hierarchy)

Trong hệ thống CLAUDE.md ba cấp, file nào KHÔNG được commit vào git?

- A. `~/.claude/CLAUDE.md` (Global).
- B. `./CLAUDE.md` (Project).
- C. `./CLAUDE.local.md` (Personal).
- D. Cả ba file đều phải commit để cả team đồng bộ.

5. (Implementation – Vòng lặp ReAct)

Trong code `agent.py` của Lab 1, dòng nào nếu **thiếu** sẽ gây infinite loop?

- A. `messages.append({"role": "tool", ...})` sau khi tool chạy.
- B. `messages.append(msg)` sau khi nhận response LLM.
- C. `json.loads(call.function.arguments)` parse arguments.

D. `return msg.content` khi `finish_reason` là "stop".

6. (Khái niệm – `/compact` vs `/clear`)

Khi context vượt 80% nhưng *vẫn đang làm cùng một task*, nên dùng lệnh nào?

- A. `/compact` – giữ lại nội dung gần nhất, tóm tắt phần cũ.
- B. `/clear` – xoá toàn bộ history, làm lại từ đầu.
- C. Không làm gì – để Claude tự xử lý.
- D. `/exit` rồi mở Claude Code lại.

7. (Khái niệm – MCP vs REST API)

Trong Lab 2 Phase 1, `tools.py` của ReAct Agent gọi Tavily qua đâu?

- A. Qua MCP server cài trong `~/.claude.json`.
- B. Qua REST API trực tiếp bằng `httpx.post`.
- C. Qua subprocess gọi Claude Code REPL.
- D. Qua Tavily Python SDK riêng (không phải `httpx`).

8. (Khái niệm – Skill auto-trigger)

Đặc trưng nào KHÔNG thuộc về Skill (so với Slash command)?

- A. Tự load khi description khớp prompt, không cần user gõ tên.
- B. Lưu ở `.claude/skills/<name>/SKILL.md`.
- C. Phù hợp cho workflow N-bước.
- D. Phải gõ `/<name>` mới invoke được.

9. (Implementation – Subagent tools)

Subagent `agent-reviewer` trong Lab 2 được giới hạn ở tool list nào?

- A. `[Read, Edit, Write]`.
- B. `[Read, Grep, Bash]` – read-only.
- C. `[Bash]` duy nhất.
- D. Không giới hạn, dùng được mọi tool.

10. (Khái niệm – Phase 4 agent-as-service)

Pattern “agent-as-a-service” trong Phase 4 (Bonus) của Lab 2 cho phép điều gì?

- A. ReAct Agent chạy như web server lắng nghe HTTP request.
- B. ReAct Agent được đóng gói thành MCP server và callable từ Claude Code session khác.
- C. ReAct Agent được deploy lên cloud và scale tự động.
- D. ReAct Agent được chuyển thành Docker container.

Phụ lục

1. **Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
3. **Github:** Các file code trên github có thể được tải [tại đây](#).
4. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 27: Hình ảnh group facebook AIO Q&A

5. Rubric:

Phần	Kiến Thức	Đánh Giá
1.	- Vibe Coding, ba vai trò Architect/Reviewer/Integrator, vị trí Claude Code trong stack Anthropic.	- Mô tả được vòng lặp Claude Code và phân biệt với chat.
2.	- Workflow E→P→C→C, Plan Mode ((Shift) + [Tab] × 2), CLAUDE.md ba cấp.	- Triển khai một commit cycle đầy đủ với Plan Mode và review diff.
3.	- Vòng lặp ReAct, OpenAI function calling, bốn pattern debug.	- Build ReAct Agent từ con số không, chạy được ba smoke test.
4.	- Context window 200K, /compact vs /clear, năm cơ chế mở rộng.	- Đo context qua /context, chọn đúng cơ chế cho từng task.
5.	- MCP vs REST API, tích hợp Tavily, subagent + skill + MCP server.	- Hoàn thành Lab 2 với cả bốn phase, quality pipeline phối hợp.

- Hết -